

Name or Value? Cue Conflict Reveals When Chain of Thought Overrides Lexical Priors

Anonymous ACL submission

Abstract

Variable names that contradict a program’s behavior degrade the accuracy of language models, but a drop in accuracy does not indicate where inside the model the name interferes. Interpretability studies of multi-step arithmetic find that intermediate values are computed during chain-of-thought (CoT) generation; in those tasks, however, the name of a variable and its computed value never disagree. We introduce a cue-conflict version of such a task, in which the same problem is presented with congruent, neutral, and incongruent variable names, and evaluate thirteen models with three sets of few-shot examples, two of them on all five difficulty levels. Without CoT, the models use the value denoted by the name: a congruent name raises accuracy by up to 20 points, and an incongruent name is produced as the answer up to 20 points above chance. With CoT, we observe no effect of the name’s value at any level, with one exception. Linear probes indicate that the lure value is decodable at the name’s own tokens and is absent at the step where the chain writes the variable’s value, and activation patching localizes the no-CoT effect to the early layers at the equation that defines the name. Whether the chain states the value does not predict the size of the effect.

1 Introduction

Consider a short arithmetic problem written with named variables:

pen=1 + cup, cup=2 + 3; pen=?

The answer is 6. We then rename the second variable to the word two:

pen=1 + two, two=2 + 3; pen=?

The arithmetic is unchanged and the answer is still 6, but the name of the variable now denotes 2, while the variable itself holds the value 5. We refer to 2 as the *lure value* of the name and to 5 as its *true value*. If a model answers 2, or uses 2 in

a later step, the meaning of the name has reached the computation.

Placing two cues in conflict in this way is the design of the Stroop task (Stroop, 1935) and of the shape–texture experiments on vision models (Geirhos et al., 2019). We apply this design to multi-step arithmetic with variable assignments.

Motivation. Names that contradict the surrounding code degrade model performance (Le et al., 2025; Lam et al., 2025; Le et al., 2026), but such results do not indicate *where* the name interferes: a drop in accuracy is consistent with a corrupted input, a corrupted computation, or a correct computation misread at the output. Kudo et al. (2026) show, with linear probes and patching on such arithmetic, that each variable’s value becomes decodable only as the CoT writes it; their variable names are single letters, so a decoded 5 can only come from the computation. Our incongruent names supply a competing value, so a probe’s output indicates which source the state holds.

We construct matched sets of problems: the same arithmetic rendered in the naming conditions of Table 1 at the five difficulty levels of Table 2, solved with and without a written chain of thought, on thirteen models at level 3 and on two of them at every level. Without CoT, the models use the value denoted by the name: a congruent name adds up to 20 points of accuracy, and an incongruent name is produced as the answer up to 20 points above chance. With CoT, we observe no effect of the name’s value at any level, with one exception among the models we test. Probes indicate that the lure value is decodable at the name’s own tokens and is absent at the step that writes the variable’s value.

2 Setup

What we take from Kudo et al. (2026). This study is built on theirs, and we state the bor-

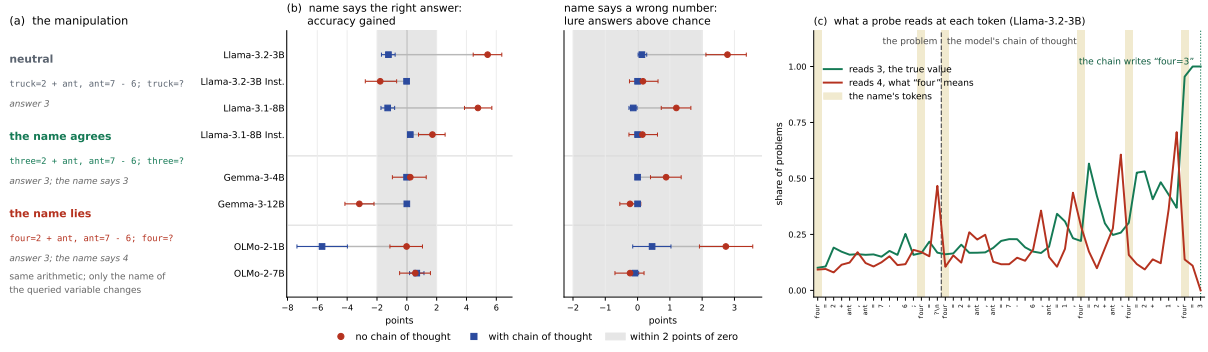


Figure 1: **(a)** One problem under three names for the queried variable. **(b)** Effect of the name at level 3, per model, mean over three demonstration sets with 95% bootstrap intervals: accuracy gained from a name denoting the answer (left), and lure answers above the neutral twin’s rate (right); grey is the two-point equivalence margin. **(c)** Per token of one problem, how often a neutral-trained probe reads the true value 3 or the name’s value 4; shaded columns are the name’s tokens.

rowing explicitly. Taken unchanged: the arithmetic task and its five levels (their Table 1), the single-digit constraint, the three fixed same-level worked examples, the probe recipe (a 10-way digit classifier on the residual stream, SGD at 10^{-3} for 10,000 full-batch epochs, one probe per token and layer), the quantities t^* , t_{eq}^* , Acc^{CoT} , Acc^{CoT} with $\tau = 0.9$, the span-by-layer-window patching design, and two of their models. Figures 2 and 4 follow the layout of their Figures 2 and 3. Ours is the cue conflict: the naming conditions, the matched twins and the rules that keep a lure from being correct for another reason, the matched-twin baselines behind every number here, the demonstration-seed claim rule and the equivalence bounds. We reimplemented their task and probes from the paper rather than running their public code, so the two code bases share none.

Task. A problem is a list of equations over the digits 0–9 with + and −, ending in a query, and every variable’s value is itself a single digit. Three worked examples of the same level precede each problem; Table 2 gives one per level. Level 3 is our main level.

Naming conditions. Each problem is generated once and rendered in the conditions of Table 1, where arithmetic, operators and digits are identical across a set and only names differ. In the *congruent* and *incongruent* conditions, exactly one variable, the *target*, is named by a number word, denoting the target’s true value in the former case and a different value, the *lure value*, in the latter. The target is either the queried or the intermediate variable, and we report the two cases separately,

since they are different manipulations.

Generation rules. One variable per problem carries a number word, and the lure value differs from every variable’s value and every digit in the problem, so a lure answer cannot be correct for another reason. Names are single tokens in every template context under each tokenizer, so twins share length and token positions (Appendix A). The *irrelevant lure* condition puts the number word on a distractor.

Regimes. In the *chain-of-thought* regime, the model writes the worked steps shown in Table 2 and then the answer. In the *direct* regime, it writes only pen=6. Our main comparison is between these two regimes.

Demonstration sets. The choice of the three worked examples affected accuracy more than any naming condition. Every behavioral result is therefore computed for three fixed demonstration sets and reported as their mean, and we count an effect as reliable only when its sign is the same in all three sets and a cluster bootstrap over matched problems excludes zero.

Probes. A linear probe reads a chosen variable’s value from the residual stream, one probe per position, layer and seed, with Kudo et al.’s recipe and hyperparameters. Probes are trained on neutral problems only, so that they never observe a number word in a name slot. Each is paired with a control probe trained on labels that are a fixed random function of the variable’s *name* (Hewitt and Liang, 2019); where the control also scores highly, the probe is reading the identity of the name rather

Condition	Problem	True	Lure
Letter	A=1 + B, B=2 + 3; A=?	—	—
Neutral	pen=1 + cup, cup=2 + 3; pen=?	—	—
Congruent	pen=1 + five , five=2 + 3; pen=?	5	—
Incongruent	pen=1 + two , two=2 + 3; pen=?	5	2
Irrelevant (level 4)	pen=1 + cup, cup=2 + 3, two =4 + 5; pen=?	9	2

Table 1: The naming conditions on one problem, whose answer is 6. *True* is the value of the variable carrying the number word and *Lure* the value that word denotes.

Level	Problem (neutral names)	Steps	Held	Distr.
1	pen=1 + cup, cup=2; pen=?	1	1	0
2	pen=2 + 3, cup=1 + pen; cup=?	2	0	0
3	pen=1 + cup, cup=2 + 3; pen=?	2	1	0
4	pen=1 + cup, cup=2 + 3, box=4 + 5; pen=?	2	1	1
5	pen=1 + cup, cup=2 + box, box=1 + 2; pen=?	3	2	0

Table 2: The five levels of Kudo et al. (2026): additions or subtractions needed (*Steps*), variables unresolved when first read (*Held*), and unused equations (*Distr.*). At level 1 the intermediate value is stated rather than computed.

than a value.

Patching. To test whether the name causes a behavior we replace the activations at its tokens with the matched twin’s and read the output again (Zhang and Nanda, 2024). Two controls accompany every intervention: one neutral word for another bounds the damage of any replacement, and one lure for another shows whether the answer follows the name.

Models. Gemma-3-12B-it, Gemma-3-4B, Gemma-3-4B-it, Llama-3.1-8B, Llama-3.1-8B-Instruct, Llama-3.2-3B, Llama-3.2-3B-Instruct, OLMo-2-1B-Instruct, OLMo-2-7B-Instruct, and four tuned checkpoints on the Llama-3.1-8B spine (Appendix C).

3 Results

Replication. On the letter condition, per-token probes trained on letters reproduce the positions reported by Kudo et al. for Llama-3.2-3B: the queried variable becomes decodable at equation 5 of the chain and the intermediate variable at equation 2, as in their Table 3.

Behavior. Table 3 reports accuracy and the pre-registered contrasts for each model and regime, averaged over three demonstration sets. Without CoT, the models use the value denoted by the

name. A congruent name on the queried variable raises the accuracy of Llama-3.1-8B by +4.8 points at level 3 and by up to 20 points at level 5, and an incongruent name is answered +1.2 points above chance at level 3 and 20 points at level 5, where accuracy is near the floor (5 to 51 points across the three demonstration sets). With CoT, these contrasts are absent. An equivalence test with a margin of 2 points finds 43 of 44 CoT cells equivalent to zero, against 12 of 44 direct cells in which an effect is present. How a model was tuned does not move this, over 6 rungs on one spine (Appendix C). The one exception is the smallest model, OLMo-2-1B-Instruct, whose CoT solves 47% of level-3 problems with word names and 77% with letters. There, a lure on the intermediate variable adds +3.2 points [+2.4, +4.0].

Does the written value explain the effect of CoT? A natural explanation is that the chain states each variable’s value, and that the stated value replaces the value the name denotes. We split the incongruent instances by whether the model’s own chain stated the target’s true value, and compare each half with the matched neutral twin under the same split. The raw lure rate differs between the halves, but so does the rate at which the twin produces that digit, since a broken chain ends in an arbitrary digit. After subtracting this baseline, the excess is +0.08 points over 132 cells in which the value was written and +0.11 over 36 in which it was not, so writing the value as text does not account for the effect. The exception survives the control: OLMo-2-1B-Instruct is lured +3.6 points [+1.9, +5.0] on the 1208 instances in which its chain wrote the correct value, and injecting the misleading name into a neutral run of it, with a correct chain force-decoded, moves the answer to the lure in 2.7% of pairs at layer 5 (7.0% of answers change in total), while the word control changes 0.4% of answers at that layer and injection moves the answer to the lure in at most 0.2% of pairs for every other model.

What does line up with the effect is whether the value is *represented* at that step. Across the 10 (model, target) cells for which we have probes, the 9 whose value is decodable where the chain writes it (accuracy ≥ 0.98) have a lure excess within 0.4 points of zero, and the one whose value is not (accuracy 0.22) is the exception.

Identifier names. A bare lexical prior is the weakest form of a misleading name, so we re-

Model	Regime	Neutral accuracy (%)	number word on the queried variable		number word on the intermediate variable	
			helps	lures	helps	lures
Gemma-3-12B-it	CoT	100 [100,100]	+0.0	+0.0	+0.0	+0.0
Gemma-3-12B-it	direct	78 [76,81]	-3.2*	-0.2	-3.7*	+0.4
Gemma-3-4B-it	CoT	100 [100,100]	-0.0	+0.0	-0.0	+0.0
Gemma-3-4B-it	direct	55 [48,59]	+0.2	+0.9*	+0.6	+0.3
Llama-3.1-8B-Instruct	CoT	100 [99,100]	+0.2	+0.0	+0.3	+0.0
Llama-3.1-8B-Instruct	direct	52 [45,65]	+1.7	+0.1	+5.2*	+0.2
Llama-3.1-8B	CoT	95 [86,100]	-1.3	-0.1	+3.8	-0.0
Llama-3.1-8B	direct	53 [41,62]	+4.8*	+1.2*	+3.8*	-0.0
Llama-3.2-3B-Instruct	CoT	100 [100,100]	-0.0	+0.0	-0.2	+0.0
Llama-3.2-3B-Instruct	direct	44 [40,50]	-1.8*	+0.2	+0.5	+0.5
Llama-3.2-3B	CoT	98 [97,100]	-1.2	+0.1	+0.9	-0.2*
Llama-3.2-3B	direct	22 [13,31]	+5.4*	+2.8*	+1.1	+1.8
OLMo-2-1B-Instruct	CoT	47 [44,51]	-5.7*	+0.4	+7.0*	+3.2*
OLMo-2-1B-Instruct	direct	34 [30,38]	-0.0	+2.7*	+3.0*	-0.2
OLMo-2-7B-Instruct	CoT	95 [93,97]	+0.7*	-0.1	-2.7	-0.0
OLMo-2-7B-Instruct	direct	54 [50,58]	+0.6	-0.2	+3.8*	-0.4

Table 3: Level 3, mean over three demonstration sets (range in brackets). *Helps*: congruent minus neutral accuracy, in points. *Lures*: lure answers minus the neutral twin’s rate of the same digit. *: same sign in all three sets and bootstrap interval excluding zero; starred CoT effects are within two points except for OLMo-2-1B-Instruct. Other levels: Table 6.

peated level 3 with identifier-style names, writing q4 where the word scheme writes four, on four models with three demonstration sets. Without CoT the digit inside the identifier is not answered: the lure excess is -0.2 to +0.2 points over 4 cells, with 0 reliable, against -0.2 to +2.8 points and 3 of 4 reliable for the same models under the word scheme. The effect is therefore specific to number words rather than to digits in a name.

Word class. One effect is not removed by CoT. At level 4, a number word on the queried variable costs Llama-3.2-3B 9.9 points whether or not the denoted value agrees, and the model then restates the wrong equation.

Probing results. Figure 1(c) shows the probe predictions at each token for Llama-3.2-3B at level 3 with CoT, Figure 2 the layer sweep, Figure 10 the same probes split by whether the model answered correctly, and Figure 5 their predictions over the model’s own wrong chains. At P2, the end of the defining equation, the true value is partly computed (accuracy 0.49, selectivity 0.32) and the lure value is weakly decodable (0.06). At P3, the query, a probe reads the lure value in 32% of instances, but the control task also scores 0.97 there, so the probe is reading the identity of the name. At P4 and P5, the true value is decodable at 0.98 and the lure is at zero in every condition and every level probed for this model.

Causal intervention. Figure 3 shows, for the direct regime, the proportion of lure errors re-

moved by patching the tokens of the name from the matched neutral twin, by layer, with both controls. For Llama-3.2-3B, patching all layers removes 52% of the lure errors and restores the margin between the true and lure values (normalized logit difference 1.00), the alternative-lure patch is followed in 9% of cases, and the span grid places the effect at the equation that defines the name, where patching layers 0–7 alone removes up to 69% of the errors. Because the word control changes 22% of correct answers at this accuracy, we report localization rather than recovery. For Llama-3.1-8B, whose no-CoT lure excess at level 3 is small, the neutral and alternative-lure patches remove lure errors equally (Figure 28), so the patching gives no evidence of a name-specific cause. With CoT, no model makes more than 1 lure error in 2,000 instances, and injection changes the answer in at most 0.2% of 500 pairs for every model except OLMo-2-1B-Instruct, reported above.

4 Related Work

Kudo et al. (2026) established when sub-answers are computed during CoT on clean arithmetic, and we add a cue conflict to that setting. Two studies apply the Stroop design to language models (Wang, 2026; Hu et al., 2026), but both measure a single next token, without a chain of thought. Faithfulness has been measured externally by perturbing prompts, chains, or parameters (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025; Tutek et al., 2025); we measure it internally under a cue whose two values are known. Since small models can copy the trailing number of the chain (Liu, 2026) and an output prior can override a correct chain (Prabhakar et al., 2024), we report a positional-copy control (Appendix B). Misleading identifiers degrade performance on real code (Le et al., 2026, 2025; Lam et al., 2025); our synthetic task allows the effect to be localized.

5 Conclusion

We introduced a cue-conflict arithmetic task to ask where a misleading variable name interferes. Without CoT its value is used as the answer, and patching localizes this to the early layers at the equation defining the name. With CoT we observe no effect at any level except in one of thirteen models, whose value is the one of 10 probed cells not decodable where the chain writes it.

Limitations

The task is synthetic arithmetic with short chains, so the results need not transfer to real code, which is the motivation for the task rather than a finding. The models are at most 12B parameters, and within-family comparisons indicate the direction of a trend only; the comparison across tuning stages rests on two families, which disagree on which stage carries the small direct-regime effect (Appendix C). Instruction-tuned models are prompted as raw completions without their chat template, which keeps token positions identical across the panel but is not how they are usually deployed. The lures are English number words; identifier-style names were tested at level 3 only, and other languages and optimized lures remain future work. Linear probes can miss information that is encoded nonlinearly; the patching experiments address this in part, but hidden states carry mixed information and patching cannot exclude side effects, which is why we report two control patches. The claim that the effect tracks whether the value is represented holds across the cells we probed, but the corresponding instance-level regression of a lure error on the probe margin is not sign-consistent (Appendix B), so that link is weaker than the cell-level pattern suggests. Finally, the CoT format writes the name beside the computed value at every step, which may itself assist recovery; the direct regime bounds this effect from the other side.

References

- Akhiad Bercovich et al. 2025. Llama-Nemotron: Efficient reasoning models. *arXiv preprint arXiv:2505.00949*.
- Yanda Chen et al. 2025. Reasoning models don't always say what they think. *arXiv preprint arXiv:2505.05410*.
- Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.
- Gemma Team. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.
- Aaron Grattafiori et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.
- Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan Huang, Aman Taxali, Alex Weigard, Richard L. Lewis, and Chandra Sripada. 2026. Conflict and congruency effects in large language models: In-weight and in-context competition in a verbal conflict task. *arXiv preprint arXiv:2608.11510*.
- Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153. Code: <https://github.com/keitokudo/faithful-cot-multistep-arithmetic>.
- Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*, volume 38. ArXiv:2504.14119.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamile Lukošiuūtė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.
- Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen. 2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.
- Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do machines struggle where humans do? LLM and human comprehension of obfuscated code. *arXiv preprint arXiv:2606.31725*.
- Ming Liu. 2026. The readout shortcut: Positional number copying dominates arithmetic CoT readout in small language models. *arXiv preprint arXiv:2605.22870*.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830.

Akshara Prabhakar, Thomas L. Griffiths, and R. Thomas McCoy. 2024. Deciphering the factors influencing the efficacy of chain-of-thought: Probability, memorization, and noisy reasoning. *arXiv preprint arXiv:2407.01687*.

Skipper Seabold and Josef Perktold. 2010. *Statsmodels: Econometric and statistical modeling with Python*. In *Proceedings of the 9th Python in Science Conference*, pages 92–96.

J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.

Team OLMo. 2024. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

Martin Tutek, Fateme Hashemi Chaleshtori, Ana Marasović, and Yonatan Belinkov. 2025. Measuring chain of thought faithfulness by unlearning reasoning steps. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Han-yu Wang. 2026. Persistent priors, preserved targets: A Stroop-style paradigm for lexical override. *arXiv preprint arXiv:2606.07555*.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. *Transformers: State-of-the-art natural language processing*. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*.

A Tokenizer checks

Our pre-CoT probe maxima on the letter condition, 30.4 and 47.9, exceed the 17.8 and 33.2 that Kudo et al. (2026) report for the same model, while the positions agree; we attribute the difference to probe training details. The same letter-trained per-token probes on Llama-3.1-8B place the intermediate variable at equation 2, as for Llama-3.2-3B, but the queried variable at equation 3 rather than 5: the equation that restates its definition (pen=1 + cup in the running example) once the intermediate’s value has been written, two equations before the one that writes the queried value. The pre-CoT maxima are 43.8 and 52.4. In the larger model the queried value is therefore decodable before the chain states it; its heatmaps and patching grids are in the figure appendix.

Rule 4 is decided per model, not by eye. For each model we tokenize every candidate name in the five contexts of the template (line start, after a comma, after each operator, before the separator) and keep only words that are a single token in all five with no merging into a neighbour. For the Llama 3 tokenizer, all ten number words pass and 95 of 112 neutral candidates pass; the lists are released with the code. The Gemma and OLMo models were run on the same lists, and the run-time guard confirmed single-token names and identical positions for them. Binary operators carry a space on each side because – otherwise merges with a following word (1–two tokenizes as 1, –two), which would break the alignment between twins. With the released lists, every instance of a condition tokenizes to the same length with the same position indices, and the runner excludes and counts any instance that does not; no instance was excluded in any run reported here.

B Full sweeps and selectivity

Irrelevant lures. At level 4 a number word on the distractor changes accuracy by at most 1.2 points. Without CoT it is nevertheless answered +2.1 to +2.3 points above the neutral baseline (2 of 2 cells reliable), so a number word naming a variable the answer does not depend on is also read as a value; with CoT the excess is +0.0 to +1.0 points (0 of 2 reliable). In the probes, a number word on a computed variable has a lure mass of 0.004 at the answer, against 0.118 for one on the distractor, whose own value is not decodable (accuracy 0.04).

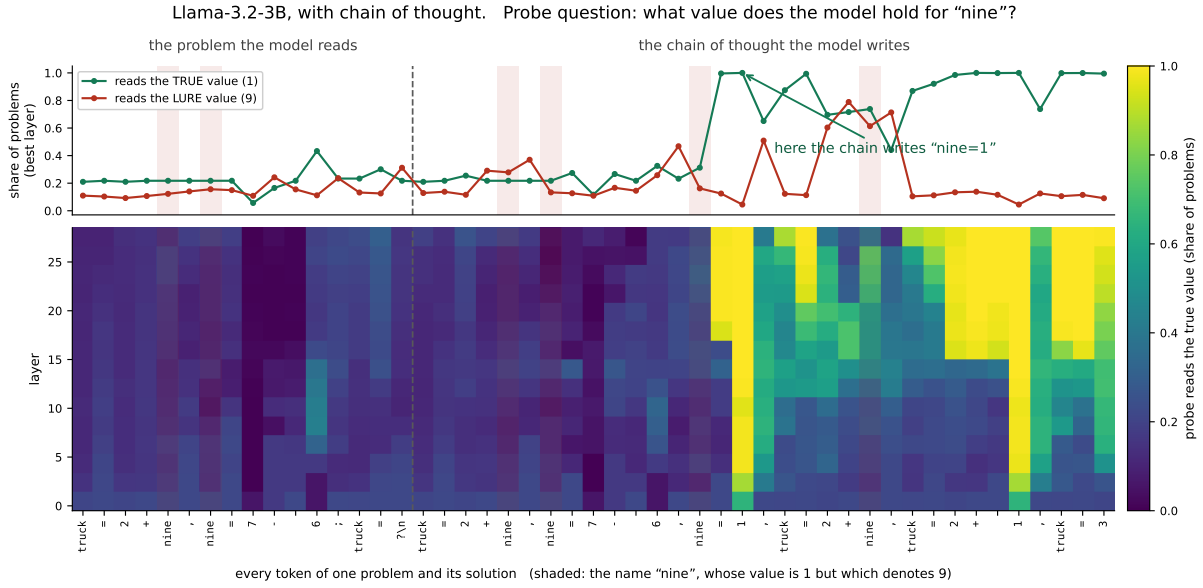


Figure 2: In the layout of [Kudo et al.](#)’s Figure 2. Per-token probes for the incongruent condition (name nine, true value 1), Llama-3.2-3B, level 3, CoT. Top: how often the best-layer probe reads the true value (green) or the lure (red). Bottom: true-value accuracy by layer. Dashed line: start of the chain; shaded: the name’s tokens.

Mixed-effects check. Every lure excess we report is a paired per-set difference summarised by a cluster bootstrap. A bootstrap over matched sets is not a model of the dependence between the two members of a twin pair, so we refit each level-3 cell as $\text{answered_lure} \sim \text{incongruent} + C(\text{seed})$ with a random intercept per matched set, on the long form of the same data: one row per set, condition and demonstration set, with the outcome on both members being the digit the misleading name denotes, so the neutral member carries the pseudo-lure baseline. The fixed effect on incongruent is the lure excess in log odds. The fit cost grows with the number of random effects, so each cell uses a fixed random subsample of 600 matched sets and both procedures are computed on exactly those rows. Of the 57 cells that admit a fit, 51 agree with the bootstrap on whether the effect excludes zero, and 11 more have no variation in the outcome and are not fitted. Of the 6 disagreements, 5 run one way: the mixed model’s interval excludes zero where the bootstrap’s does not, and 4 of these are negative, meaning fewer lure answers under the misleading name than under its neutral twin. In 1 the mixed model withdraws support for an effect the bootstrap finds (Gemma-3-12B-it, value-only chain, queried variable); none of these is a cell of Table 3, so no claim in the body rests on the choice of procedure. We keep the bootstrap in the body because it estimates the

quantity in Table 3 on the scale it is reported.

Instance-level link, and where it fails. The body argues at the level of cells: the one cell whose value is not decodable where the chain writes it is the one cell with a surviving effect. The finer version of that claim is per instance, so we also fit, for each (model, level, regime, role), a clustered logit of a lure error on the probe’s margin ($\log p_{\text{true}} - \log p_{\text{lure}}$) at the layer with the best neutral accuracy. The mechanism predicts a negative coefficient: a state that favours the true value should produce fewer lure answers. Across 57 fits, 27 reach $p < .05$, of which 22 are negative and 5 positive. The predicted direction dominates but is not uniform, and 4 of the exceptions come from a single model, so we treat the instance-level link as unresolved and rest the claim on the cell-level pattern.

Lure index. The shape–texture literature reports a cue-conflict index rather than a difference: lure answers over lure plus true answers, with congruent trials excluded ([Geirhos et al., 2019](#)). Over the 18 (model, target) cells at level 3 it is 0.02–0.38 without CoT and 0.00–0.16 with it. We report the twin-subtracted excess in the body instead, because the index has no baseline for the rate at which a model produces that digit anyway, which at these accuracies is most of it.

Positional-copy control. Liu (2026) report that small models copy whichever number stands in the trailing slot of the chain. Pooling the 4073 CoT lure errors at the answer position over every model, level and demonstration set, the number immediately preceding the answer in the model’s own chain is the lure in 13% of them and the target’s true value in 11%. Neither dominates, so positional copying does not account for the CoT lure errors that remain.

For every (variable, position, layer, seed) we report probe accuracy for the true value, lure rate, lure mass and margin, alongside the accuracy of a control probe trained on labels that are a deterministic random function of the variable’s name (Hewitt and Liang, 2019). Selectivity is the difference. The query position is the one cell where a high lure rate coincides with control accuracy near 1, which is why it is reported as name identity rather than as a represented value. Figure 2 gives the per-token sweep for the incongruent condition with CoT; the sweeps for the other conditions and the direct regime, with t^* , t_{eq}^* and the pre-/post-chain accuracies in Kudo et al.’s format, are released with the code.

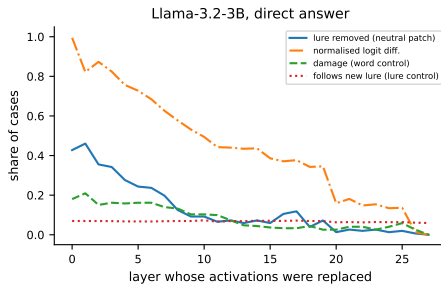


Figure 3: Lure errors removed by patching the name’s tokens from the neutral twin, by layer, direct regime, with the neutral-word control patch (damage) and the alternative-lure patch.

C How the models were tuned

Table 3 varies family and size. It does not vary how a model was produced from its pretrained weights, so we add a ladder that holds the spine fixed at Llama-3.1-8B-Instruct and changes only the tuning: the pretrained model, the instruction-tuned model, a single-task LoRA finetune, a joint finetune over six tasks, a TIES merge of six single-task adapters, and a reasoning-tuned checkpoint on the same spine. The three LoRA rows are folded into the weights before evaluation, so ev-

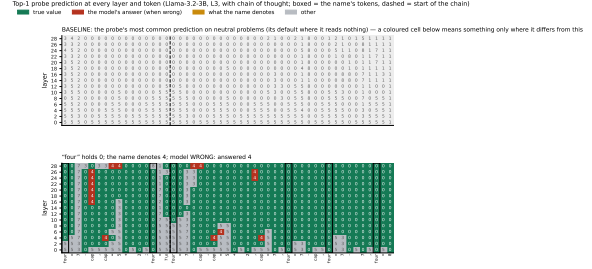


Figure 4: In the layout of Kudo et al.’s Figure 3, for problems this model answers with the lure. Each cell is the probe’s top-1 prediction at that layer and token: green the variable’s true value, red the value the model answered, amber the value its name denotes, grey anything else. The first panel is the probe’s most common prediction on neutral problems, which is what it emits where it reads nothing; cells carry information only where they differ from it.

ery model in the ladder is an ordinary causal LM with the same tokenizer; the verified word pool, the token positions and the matched sets are therefore identical down the ladder, and each rung is comparable to the others instance by instance.

Tuning	Model	Queried	Intermediate	Acc.
base	Llama-3.1-8B	+1.20*	-0.02	53.4
instruct	+ instruct	+0.15	+0.22	52.2
single-task finetune	+ task LoRA	+0.05	+0.23	52.2
multi-task finetune	+ multi-task LoRA	-0.58*	-0.08	46.1
weight merge	+ TIES merge	-0.10	+0.10	51.9
reasoning-tuned	Nemotron-Nano-8B	+0.52	+0.32	34.5
base	Gemma-3-4B	+0.08	+0.02	60.2
instruct	+ instruct	+0.88*	+0.33	54.9

Table 4: Level 3, no CoT, lure answers above the matched twin’s rate, mean over three demonstration sets. Acc. is neutral accuracy. * as in Table 3. The bottom block repeats the pretrained-to-instruct step in a second family, Gemma-3-4B.

Without CoT the pretrained model is the only rung of the Llama ladder with a reliable positive effect (+1.20 on the queried variable); the instruction-tuned model is at +0.15, and the three tuned variants lie between -0.58 and +0.23, none differing from the instruction-tuned model by more than 0.73 points over 6 cells. With CoT every rung is within 0.30 of zero. Two numbers in the table need their context. The multi-task finetune’s -0.58 is reliable and negative, and it is also the only rung that loses accuracy (46.1% against 52.2%), so the likelier reading is that the joint finetune is worse at the task rather than more robust to the name. The reasoning-tuned model has the lowest accuracy of the six (34.5%), and a model that answers arbitrarily produces the lure digit and the

Top-1 probe prediction over the model's own chain (Llama-3.2-3B, L3, with chain of thought; probes trained at the value step of "v1" on neutral problems; 8 instances)

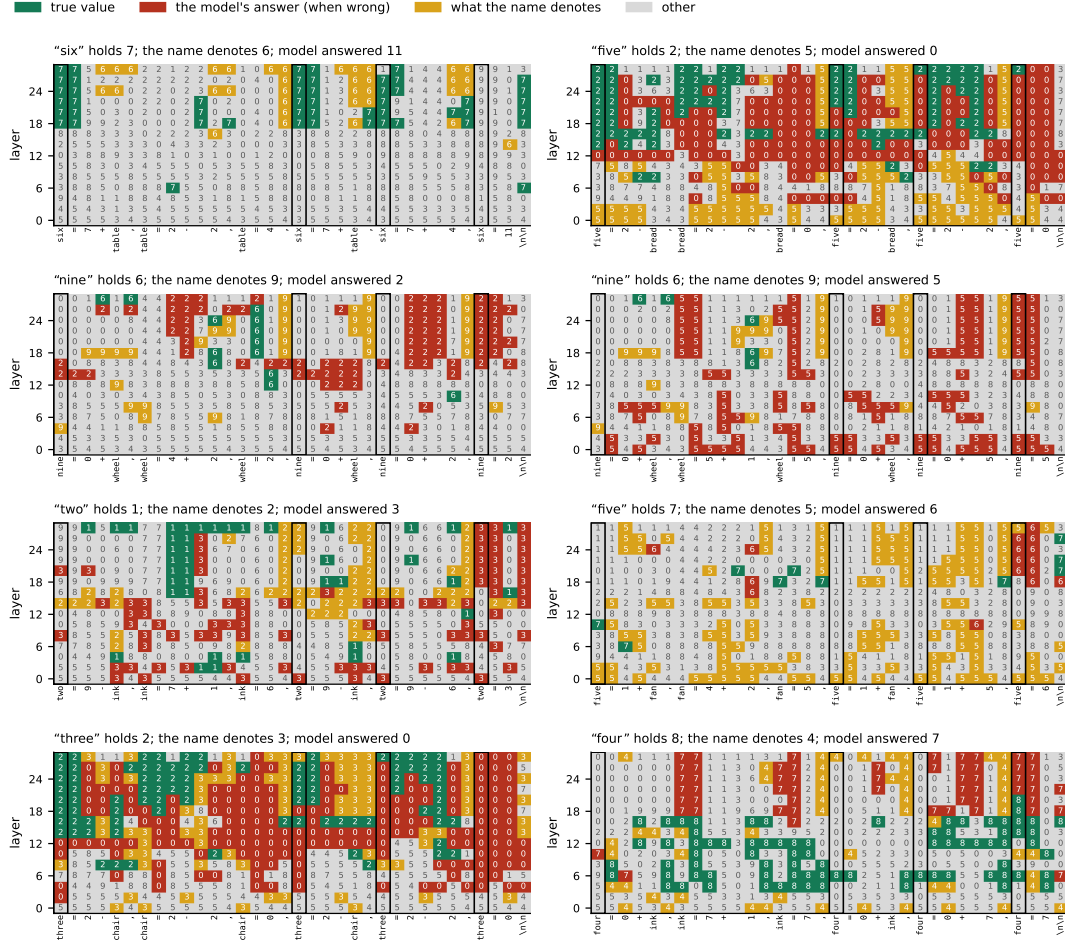


Figure 5: The probes of Figure 4 applied to the model’s own generated chain rather than to a force-decoded gold chain, for the 8 level-3 incongruent problems that Llama-3.2-3B answered wrongly under CoT (the first 8 of 108 wrong answers in 2,000 instances). Probes are trained on neutral problems at the step that writes the variable’s value and applied at every generated token and every second layer; colours as in Figure 4, boxed columns the name’s own tokens. With 8 instances this is a picture, not a rate: the states hold the value the model is about to write (red) in the late layers, and the name’s value (amber) is confined to the early layers at and after the name’s tokens.

twin’s digit alike, which is what the twin subtraction is for. It is also the only rung whose CoT lure rate is above zero (+0.30 on the intermediate variable), far inside the equivalence margin and not reliable under our rule, and we report it rather than claim it.

This checkpoint gates its long-form reasoning on a system instruction. Our prompts are raw completions with no system message, which is what keeps token positions identical across the panel, so this rung is a reasoning-tuned model with that mode off. A model reasoning at length would need a chat template and a different token layout, and is outside this comparison.

The Gemma pair in the bottom block runs the

other way: the pretrained model has no reliable effect (+0.08 on the queried variable) and the instruction-tuned model has one (+0.88). Which tuning stage carries the small direct-regime effect therefore differs between families, and the ladder should not be read as showing that the effect comes from pretraining in general.

D Probe recipe and patching scope

Two choices inherited from Kudo et al. (2026) could in principle carry our probing and patching results, so we vary each one.

Recipe. We refit the level-3 probes for Llama-3.2-3B with L-BFGS, and with SGD on standard-

ized inputs, against the reported recipe.

Position	Variable	SGD (reported)	L-BFGS	standardized
defining equation	queried	0.184	0.190	0.183
defining equation	intermediate	0.496	0.422	0.420
query	queried	0.183	0.316	0.280
query	intermediate	0.271	0.366	0.393
value step	queried	1.000	1.000	1.000
value step	intermediate	0.982	0.962	0.972
answer	queried	1.000	1.000	1.000

Table 5: Best-layer accuracy for the true value, neutral instances, under three probe recipes.

The picture is the same under all three: the value is decodable where the chain writes it (0.96–1.00) and not at the end of its defining equation (0.18–0.50). The query position moves, from 0.271 under the reported recipe to 0.393 under standardization, within a range of 0.18–0.39 over recipes and variables. We already report that position as the presence of the name rather than a decoded value, because its control task scores 0.97, so a recipe that reads the name better does not change its reading.

Scope. Patching every occurrence of the name includes the copy the model writes in its own output. Restricting the patch to the name’s occurrences in the prompt leaves the intermediate variable unchanged, since its name never appears in the answer: removal is 43% at layers 0–3 under both scopes. For the queried variable the two agree from layer 4 on (52% at layers 4–7, against 53% at layers 0–3) and disagree at layer 0, where the prompt-only patch removes 3% and damages 76% of correct answers. That is mechanical: the queried variable’s name also sits at the position we read, so patching its prompt copies alone leaves the model reading a name whose early representation no longer matches. The localization we report does not depend on the choice.

E A chain that writes values without equations

Our chain restates each equation before substituting into it, as Kudo et al.’s does. Their Table 6 also reports a format that writes only the sub-results, cup=5, pen=6. That format is the arithmetic counterpart of a code trace, which writes a variable’s value and never the expression that produced it, so we ran it on the panel at level 3 with three demonstration sets, with two predictions registered in advance: that the name would reach the value step, and that the value would still be decodable there, making any failure one of readout.

Both predictions fail. Llama-3.1-8B writes the lure at the intermediate’s value step +0.15 points above its twin, inside the equivalence margin; Llama-3.2-3B is at +9.20, but across the three demonstration sets that is +1.7 / +24.2 / +1.7, so one set carries it. More informative is what the probes say. At the step that writes the value, the true value is decodable at 0.81 and 0.59 for Llama-3.1-8B and at 0.61 and 0.79 for Llama-3.2-3B, against 1.00 and 0.99 for the same model under the full chain. Neutral accuracy falls with it, to 54% and 31%.

Dropping the equations does not leave the computation intact and expose the readout. It degrades the computation. Whatever a terse trace does to a model reading code, the arithmetic analogue of that format is a different phenomenon, and we do not treat the two as one.

F Additional levels, models and demonstration sets

Table 6 repeats Table 3 for the other four levels. The pretrained Gemma-3-4B, the base half of the Gemma pair, is reported in Table 4 rather than in Table 3.

Data, compute and software. Each level has 2,000 matched sets, each rendered in every condition, and 10,000 separately generated problems for training probes; every probe is trained with 3 seeds. Every run is inference: no language model is trained, and the probes are linear. All experiments together used at most 178 GPU-hours on NVIDIA A30, H100 and H200 cards, counted as allocated time and so including idle time of the job queue. The models are from the Llama 3 (Grattafiori et al., 2024), Gemma 3 (Gemma Team, 2025) and OLMo 2 (Team OLMo, 2024) families, with the reasoning-tuned checkpoint from Bercovich et al. (2025), each used under its release licence for research. Models run in PyTorch (Paszke et al., 2019) with Transformers (Wolf et al., 2020), and the probes are trained in PyTorch; the L-BFGS refit of Appendix D uses scikit-learn (Pedregosa et al., 2011), and the mixed-effects check uses statsmodels (Seabold and Perktold, 2010).

Per-set values, the pooled estimate and the claim rule are released with the code, together with the span-by-layer patching grids for all three sources, the level-4 irrelevant-lure control, the lure-distance analysis, the teacher-forced lure

Model	Level	Regime	Neutral accuracy (%)	number word on the <i>queried</i> variable		number word on the <i>intermediate</i> variable	
				helps	lures	helps	lures
Llama-3.1-8B	1	CoT	99 [98,100]	+0.0	+0.0	+0.3	+0.0
Llama-3.1-8B	1	direct	74 [38,99]	+16.1*	+11.2*	+8.8*	+1.3
Llama-3.2-3B	1	CoT	97 [95,100]	+1.1	+0.1	-2.4	+0.3
Llama-3.2-3B	1	direct	46 [21,93]	+0.6	+5.9*	-3.9*	+2.6*
Llama-3.1-8B	2	CoT	100 [100,100]	-0.1	+0.0	-0.0	+0.0
Llama-3.1-8B	2	direct	36 [33,37]	+11.8*	+0.7	+2.6	+2.2
Llama-3.2-3B	2	CoT	94 [90,98]	+3.7	+0.5*	+1.0*	-0.1
Llama-3.2-3B	2	direct	29 [27,30]	+10.8*	+2.5*	+2.6*	+2.2
Llama-3.1-8B	4	CoT	88 [63,100]	+1.0	-0.0	-2.3	+0.2
Llama-3.1-8B	4	direct	28 [26,31]	+13.7*	+5.8	+6.0*	+0.2
Llama-3.2-3B	4	CoT	64 [26,85]	-10.2*	+0.7	-2.6*	+0.4
Llama-3.2-3B	4	direct	20 [16,23]	-1.6	+1.9	-0.2	+1.2*
Llama-3.1-8B	5	CoT	65 [43,100]	+1.2	-0.3	+0.1	-0.1
Llama-3.1-8B	5	direct	20 [13,26]	+20.2*	+20.5*	+0.9	+0.4
Llama-3.2-3B	5	CoT	69 [48,97]	-1.4	+0.1	-2.5	-0.1
Llama-3.2-3B	5	direct	12 [11,12]	+4.7*	+3.5*	+1.1*	+0.5

Table 6: As Table 3, for levels 1, 2, 4 and 5. Level 1 is the no-computation control: the intermediate variable’s value is stated rather than computed.

rates, and the error taxonomy behind the word-class effect.

G Every figure at full size

Each figure below is reproduced on its own page. Wide figures are placed on landscape pages, which a PDF viewer displays upright, so nothing has to be read sideways.

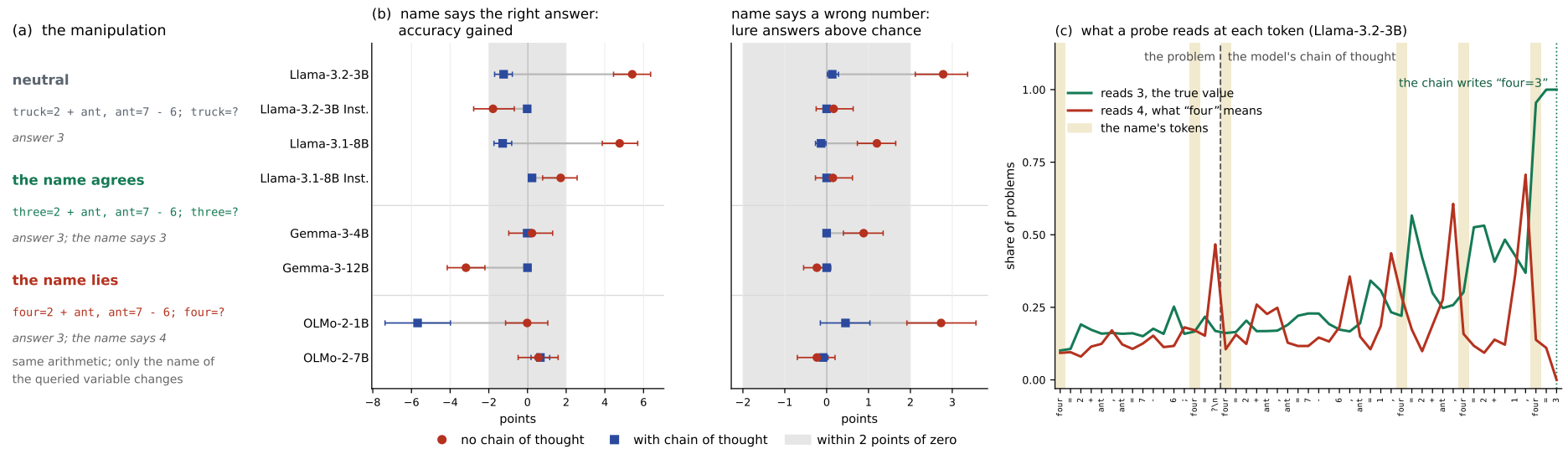


Figure 6: Figure 1 at full size.

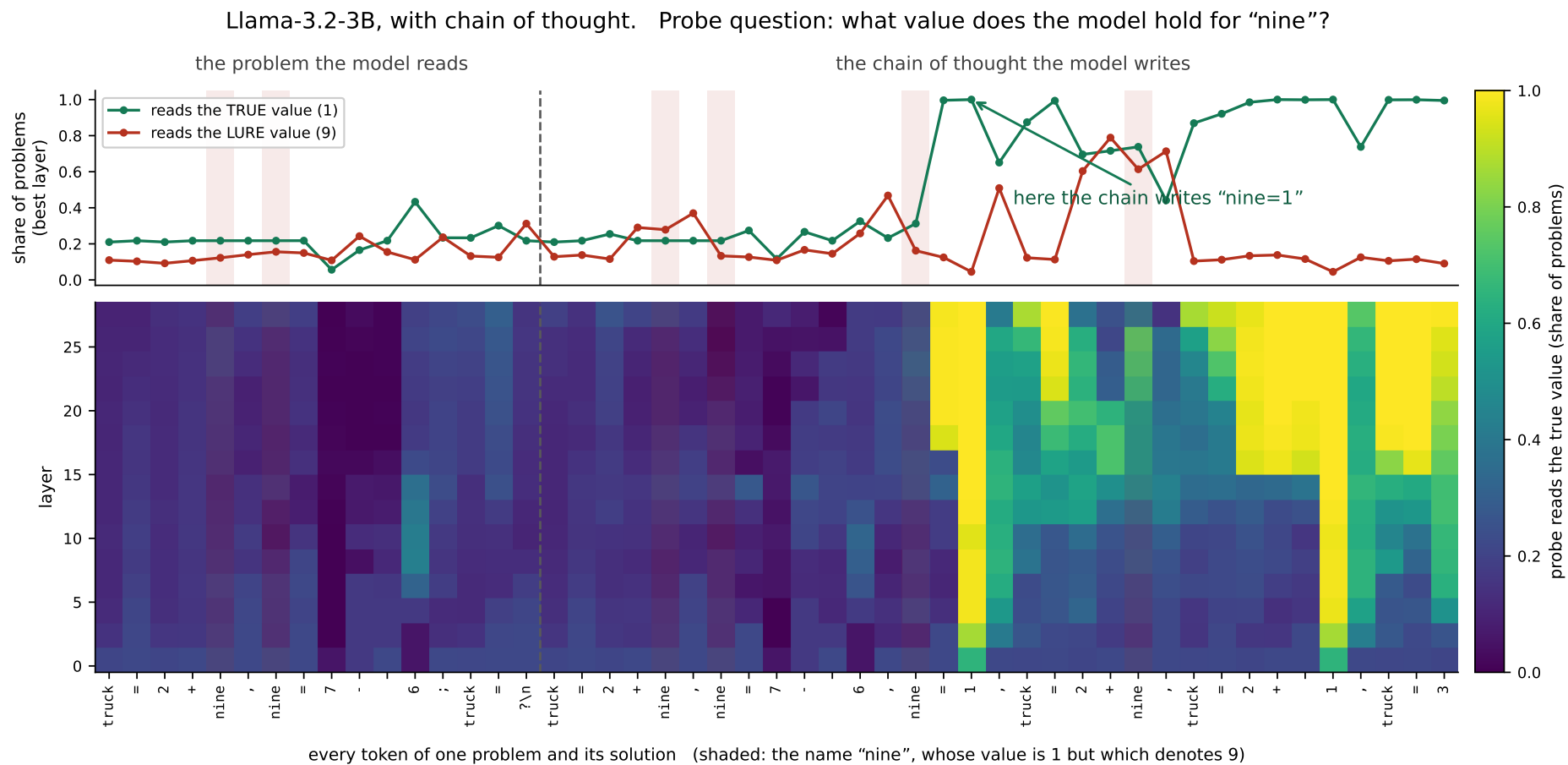


Figure 7: Figure 2 at full size: per-token probes, chain-of-thought regime, intermediate variable.

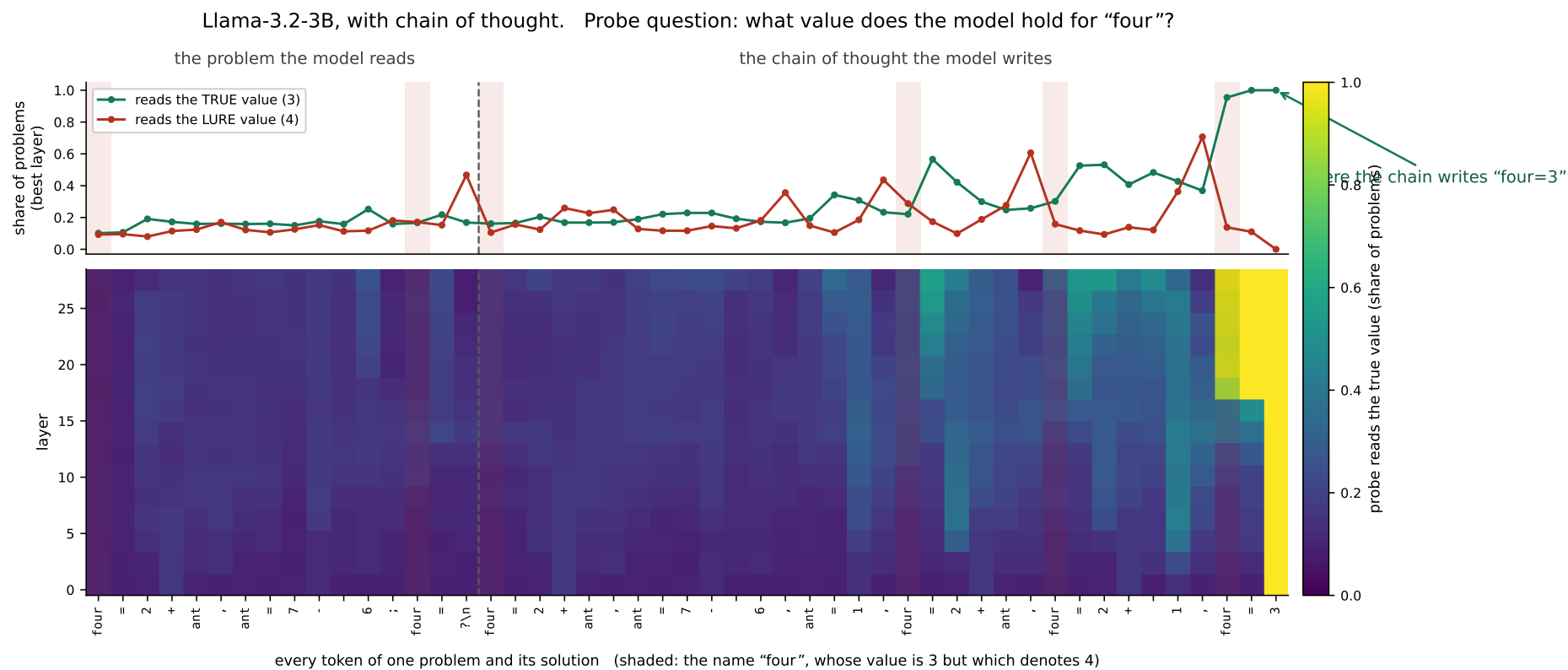


Figure 8: Per-token probes, chain-of-thought regime, queried variable.

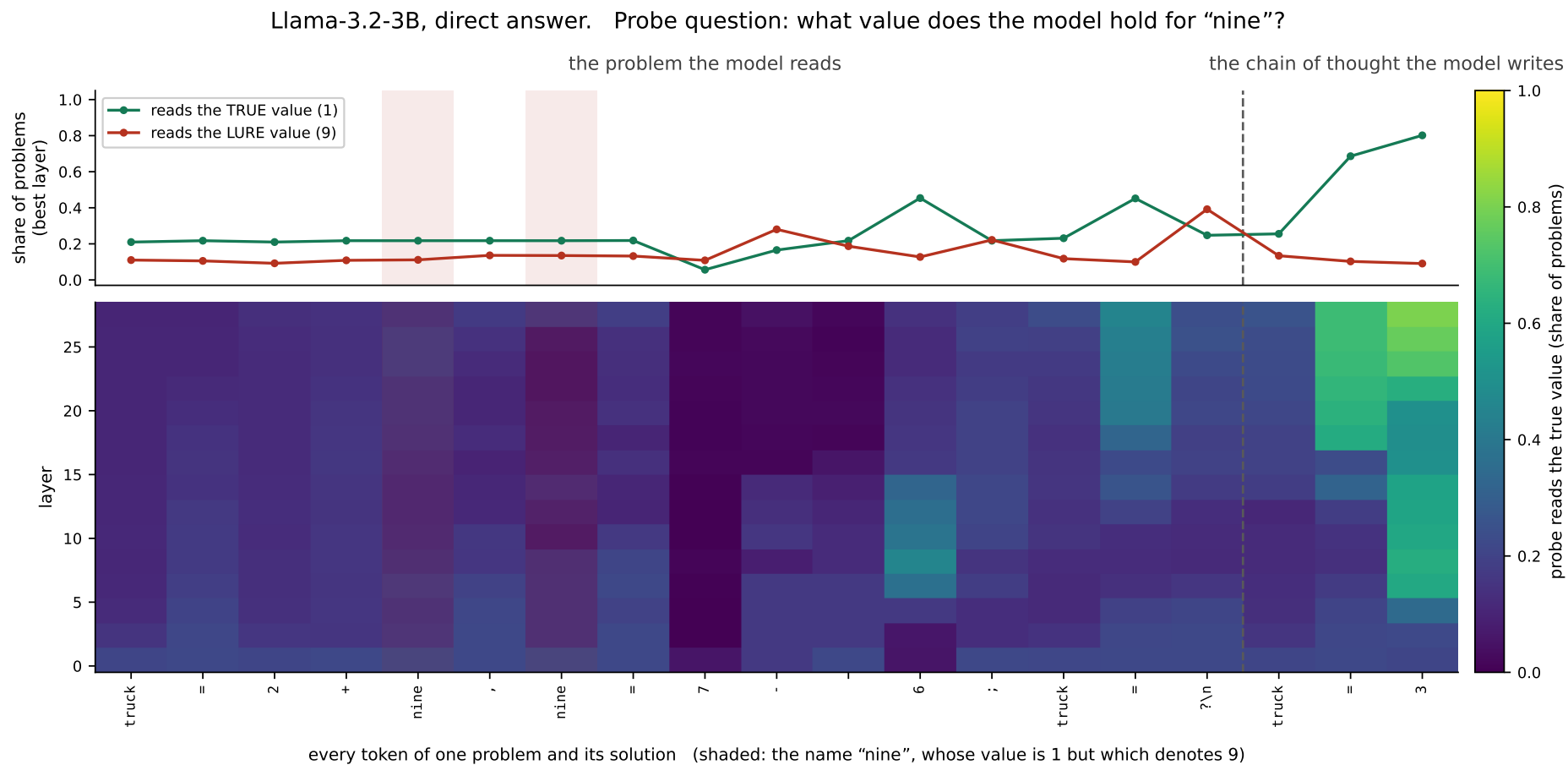
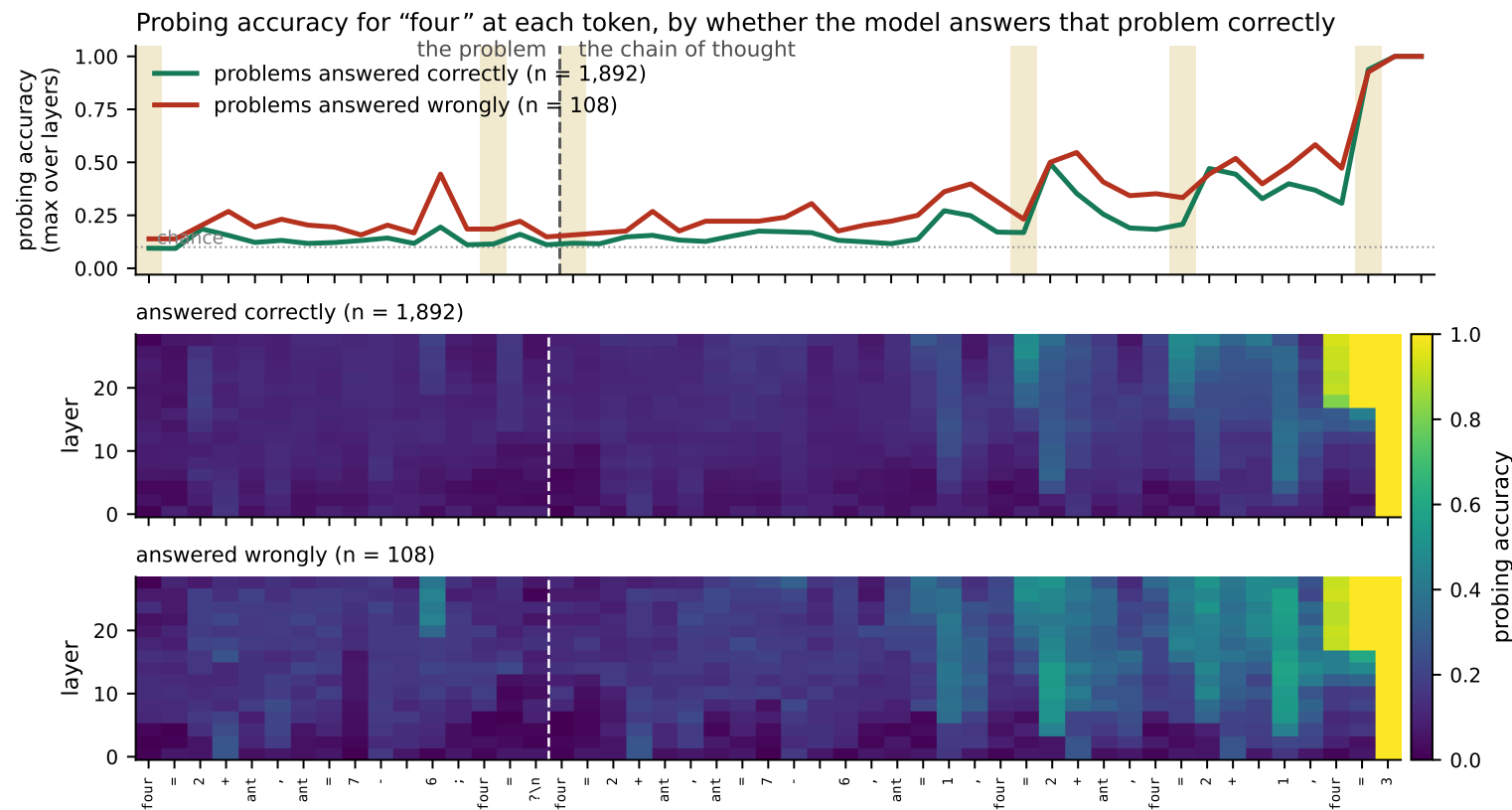


Figure 9: Per-token probes, direct regime, intermediate variable.



Probes read the variable's true value; states come from a forced pass over the correct chain, so the lower panel asks what the state holds for problems this model gets wrong unaided.

Figure 10: Probing accuracy split by whether the model answers the problem correctly on its own, with the layer sweep for each half.

Top-1 probe prediction at every layer and token (Llama-3.2-3B, L3, with chain of thought; boxed = the name's tokens, dashed = start of the chain)

■ true value ■ the model's answer (when wrong) ■ what the name denotes ■ other

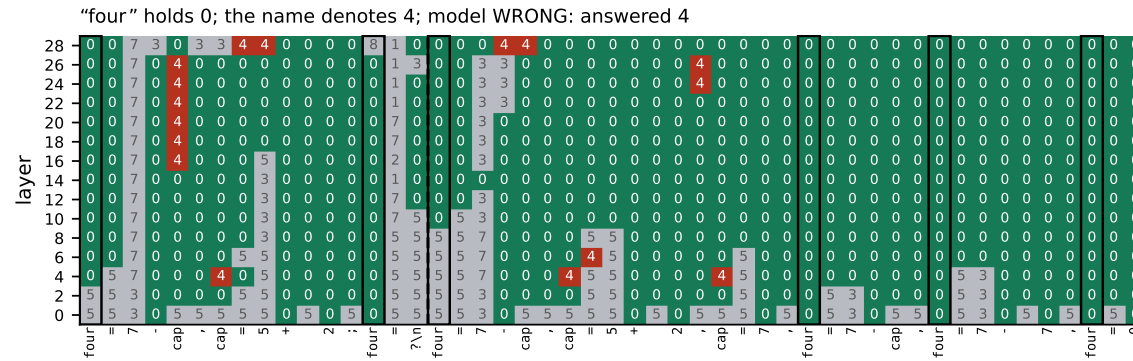
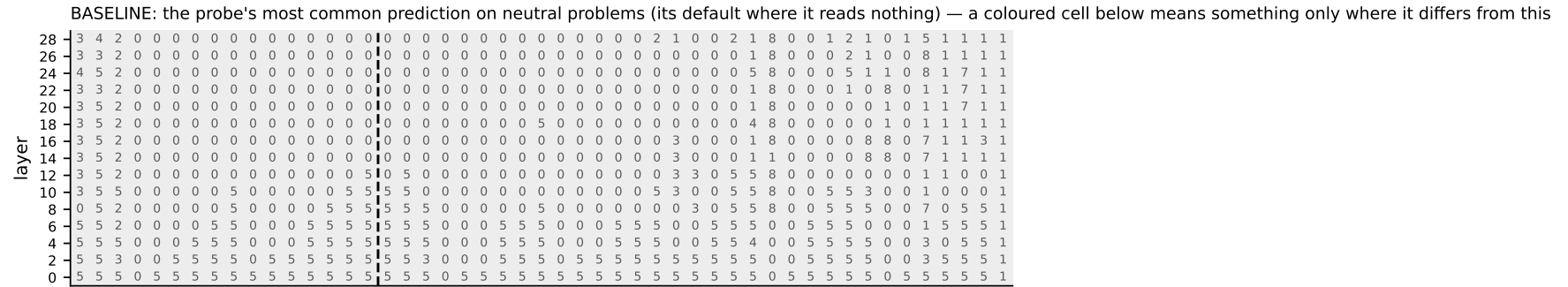


Figure 11: Top-1 probe prediction at every layer and token, on a problem answered with the lure. Layout after Kudo et al.'s Figure 3.

Top-1 probe prediction at every layer and token (Llama-3.2-3B, L3, with chain of thought; boxed = the name's tokens, dashed = start of the chain)

■ true value ■ the model's answer (when wrong) ■ what the name denotes ■ other

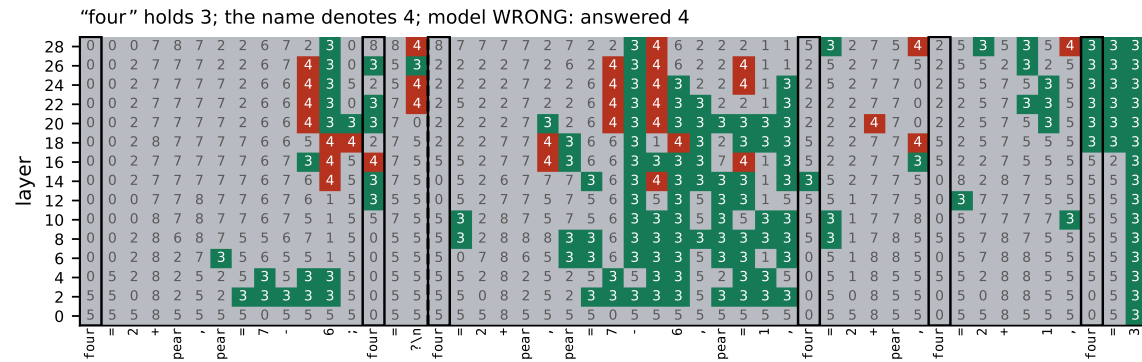
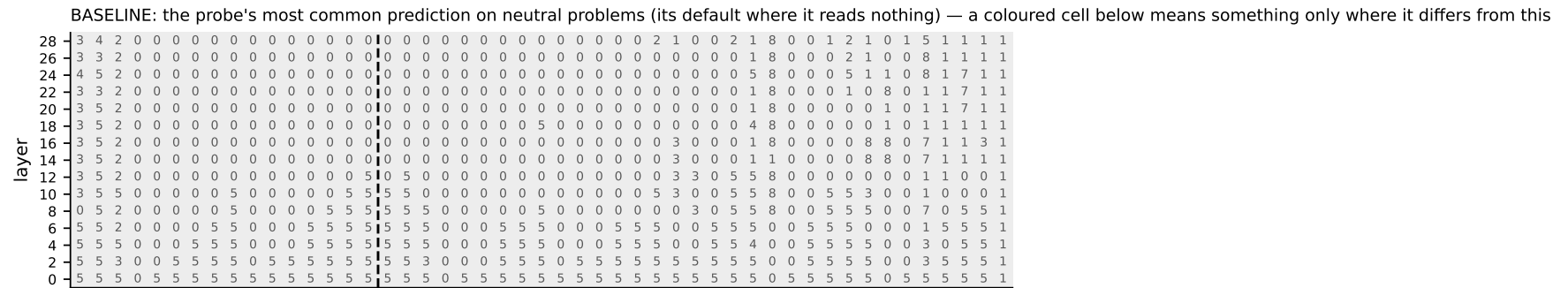


Figure 12: A second problem answered with the lure.

Top-1 probe prediction at every layer and token (Llama-3.2-3B, L3, with chain of thought; boxed = the name's tokens, dashed = start of the chain)

■ true value ■ the model's answer (when wrong) ■ what the name denotes ■ other

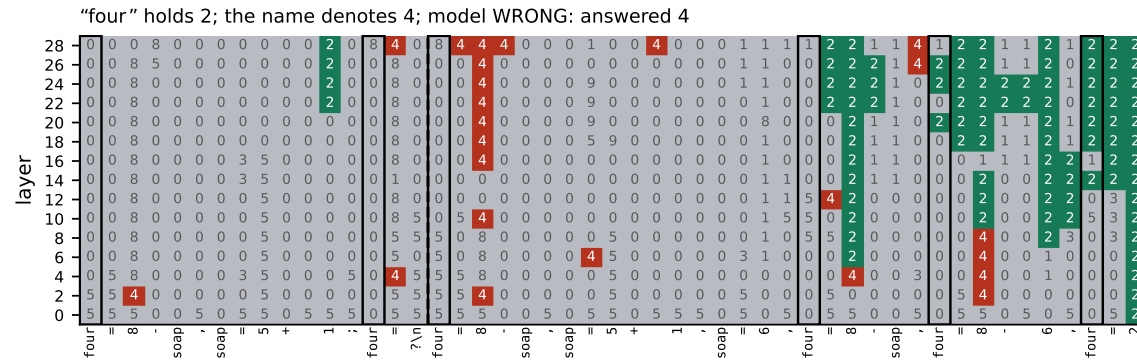
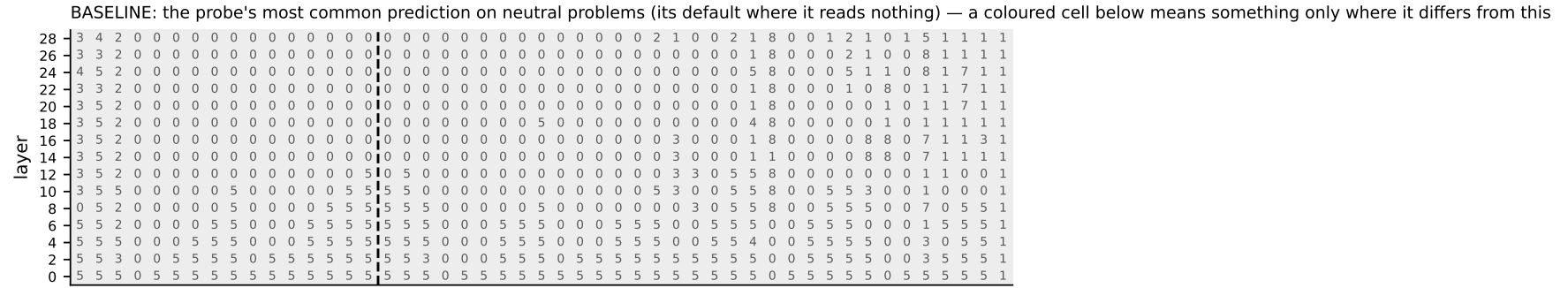


Figure 13: A third problem answered with the lure.

21

[illegible]

“six” holds 7; the name denotes 6; model WRONG: answered 11

28
26
24
22
20
18
16
14
12
10
8
6
4
2
0

layer

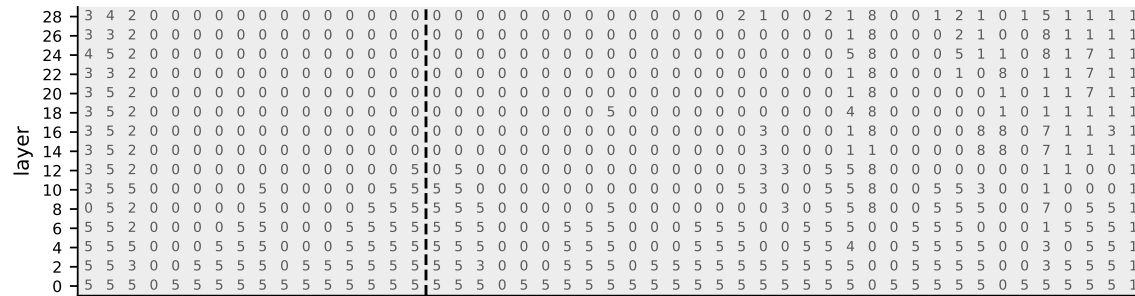
six 7 = + table 6 2 - 2 : six ? six 7 + table table 2 - - 2 table 0 = six = 7 + table six = 7 + table 0 = six = 7

Figure 14: As above, on a problem answered wrongly for another reason.

Top-1 probe prediction at every layer and token (Llama-3.2-3B, L3, with chain of thought; boxed = the name's tokens, dashed = start of the chain)

true value
 the model's answer (when wrong)
 what the name denotes
 other

BASELINE: the probe's most common prediction on neutral problems (its default where it reads nothing) — a coloured cell below means something only where it differs from this



"five" holds 2; the name denotes 5; model WRONG: answered 0

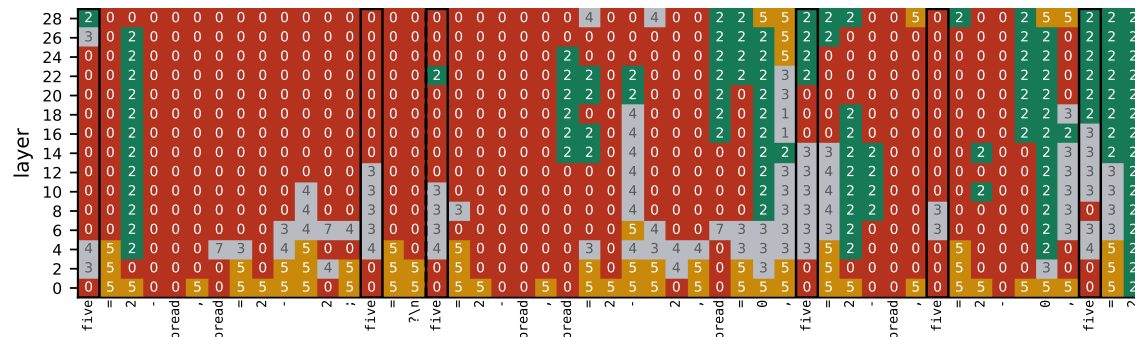


Figure 15: A second such problem.

Top-1 probe prediction at every layer and token (Llama-3.2-3B, L3, with chain of thought; boxed = the name's tokens, dashed = start of the chain)

■ true value ■ the model's answer (when wrong) ■ what the name denotes ■ other

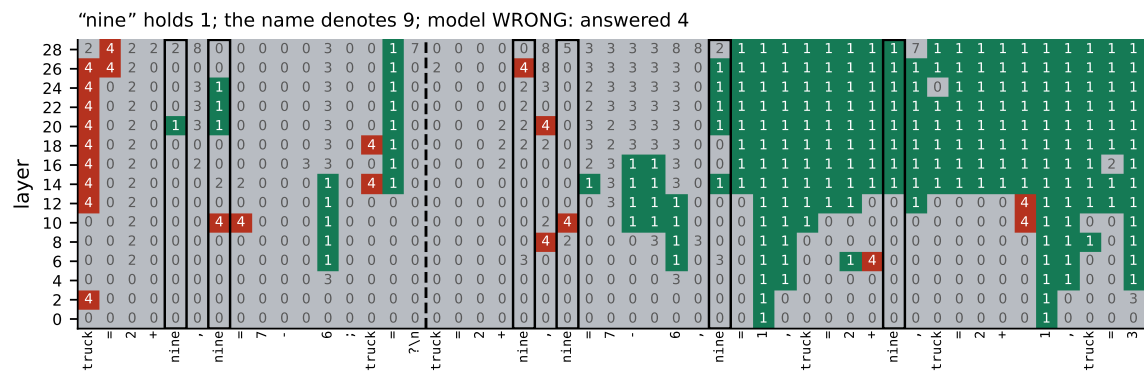
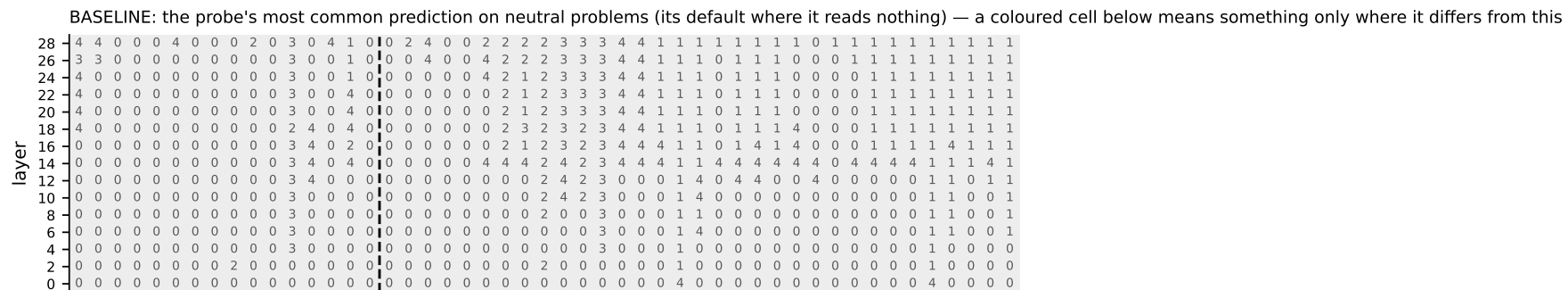


Figure 16: As above, for the intermediate variable.

Top-1 probe prediction at every layer and token (Llama-3.2-3B, L3, with chain of thought; boxed = the name's tokens, dashed = start of the chain)

■ true value ■ the model's answer (when wrong) ■ what the name denotes ■ other

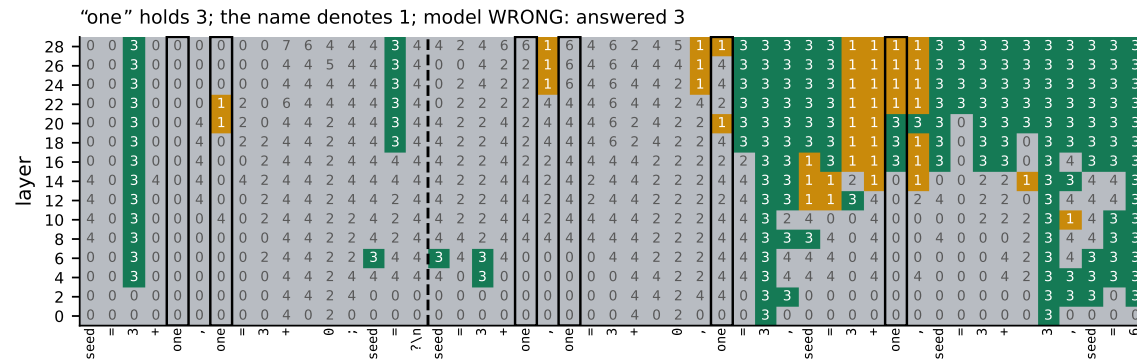
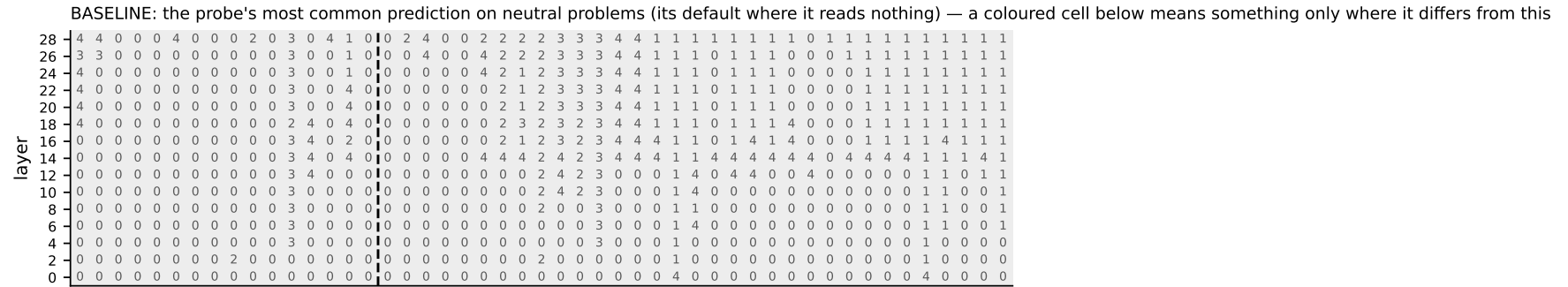


Figure 17: A second problem, intermediate variable.

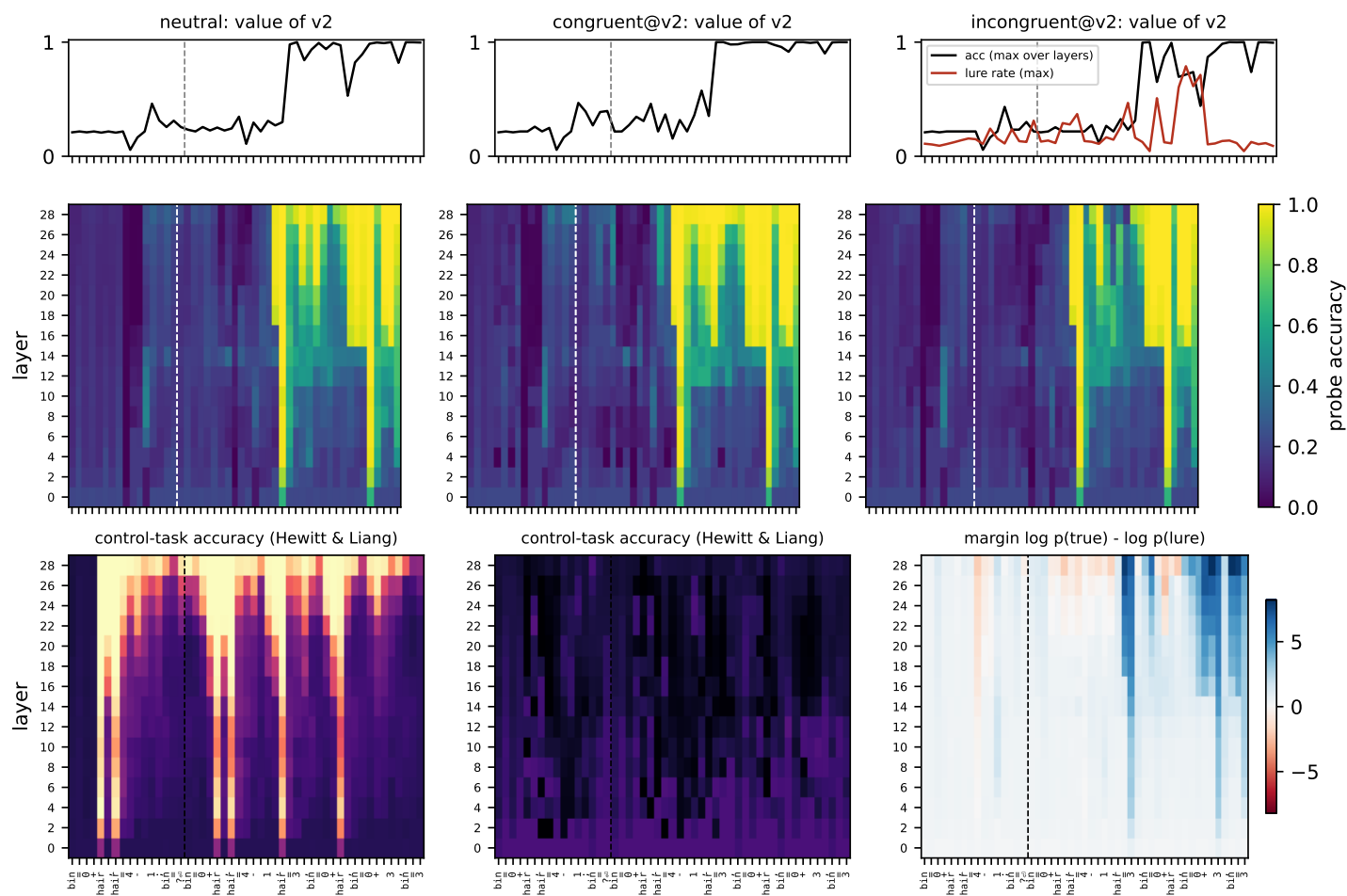


Figure 18: Accuracy heatmaps by token and layer for the neutral, congruent and incongruent conditions, in the layout of Kudo et al.’s Figure 2.

llama32-3b, level 3, direct: per-token probes for the value of v2 (trained on train_letter)

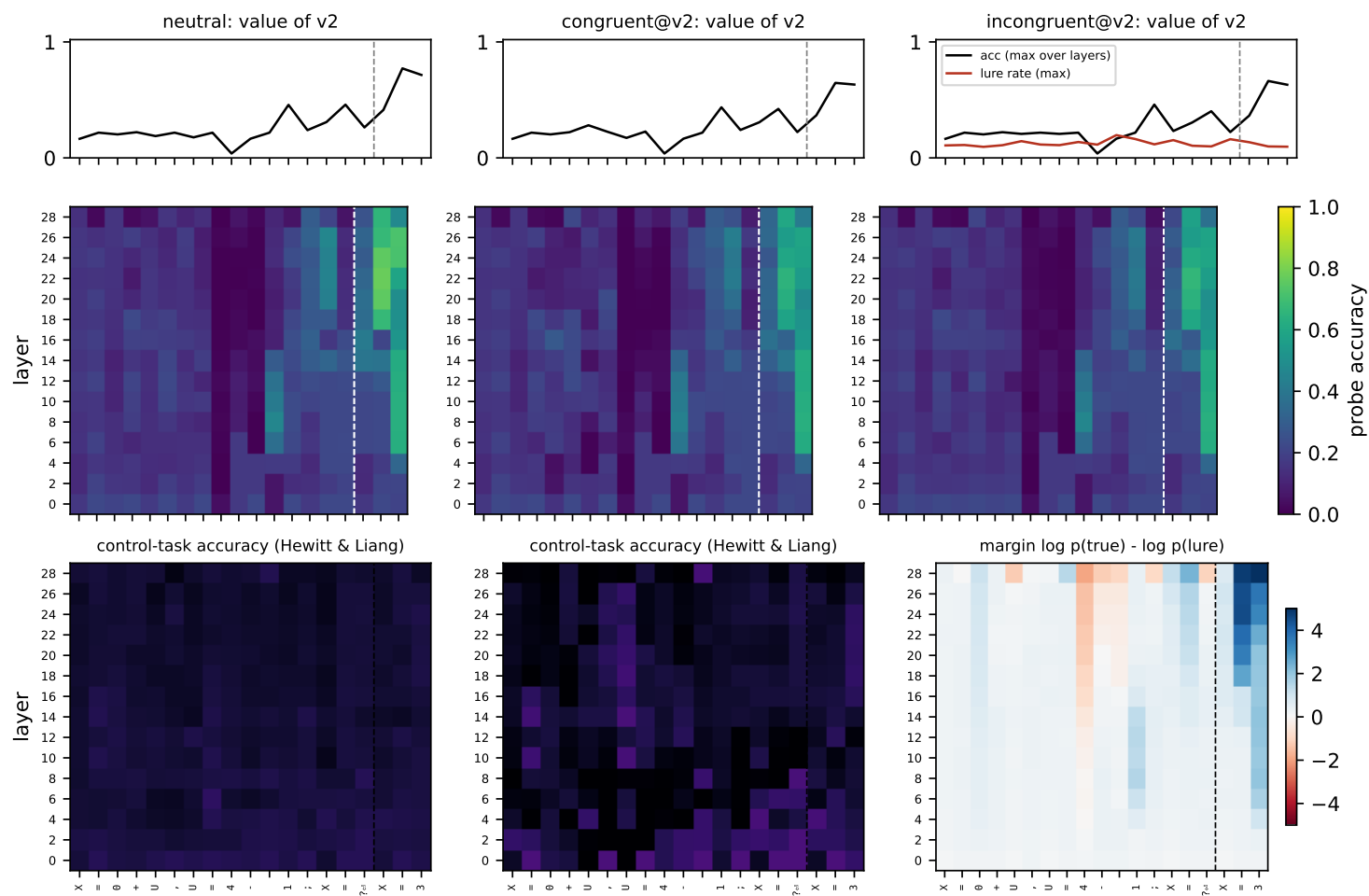


Figure 19: The same heatmaps in the direct regime.

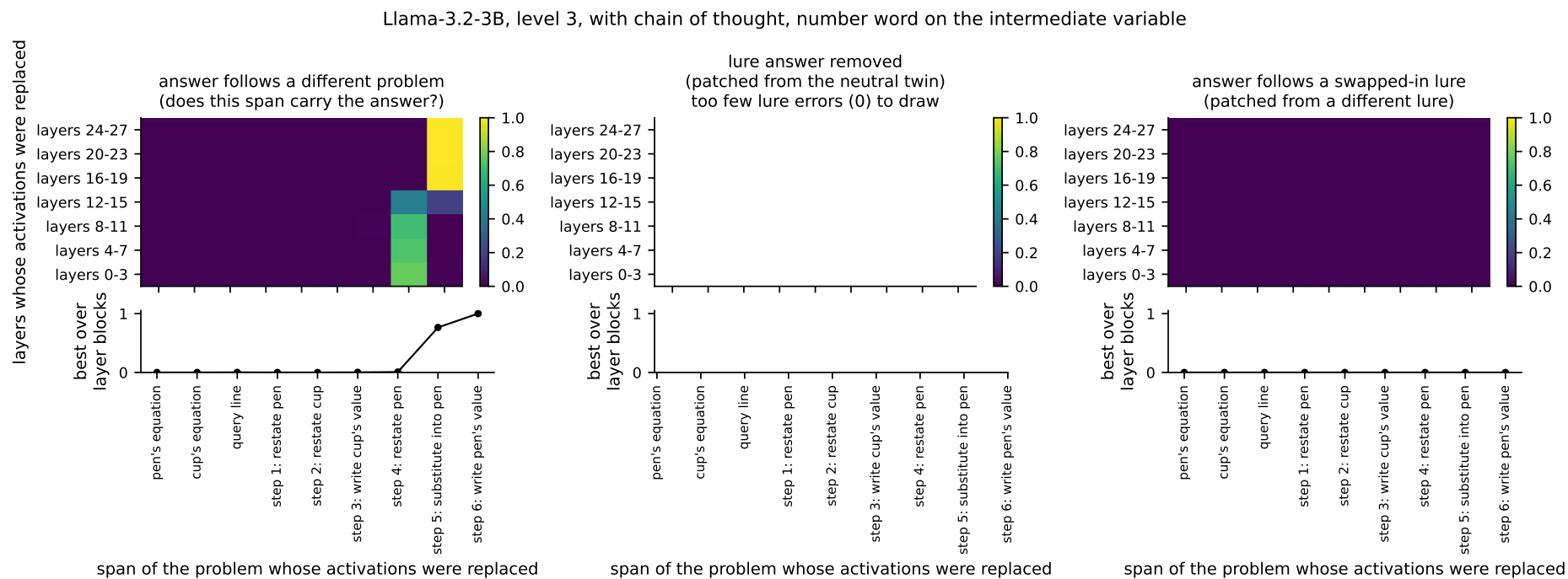


Figure 20: Span-by-layer-window patching grids, three sources, after Kudo et al.'s Figures 5–6.

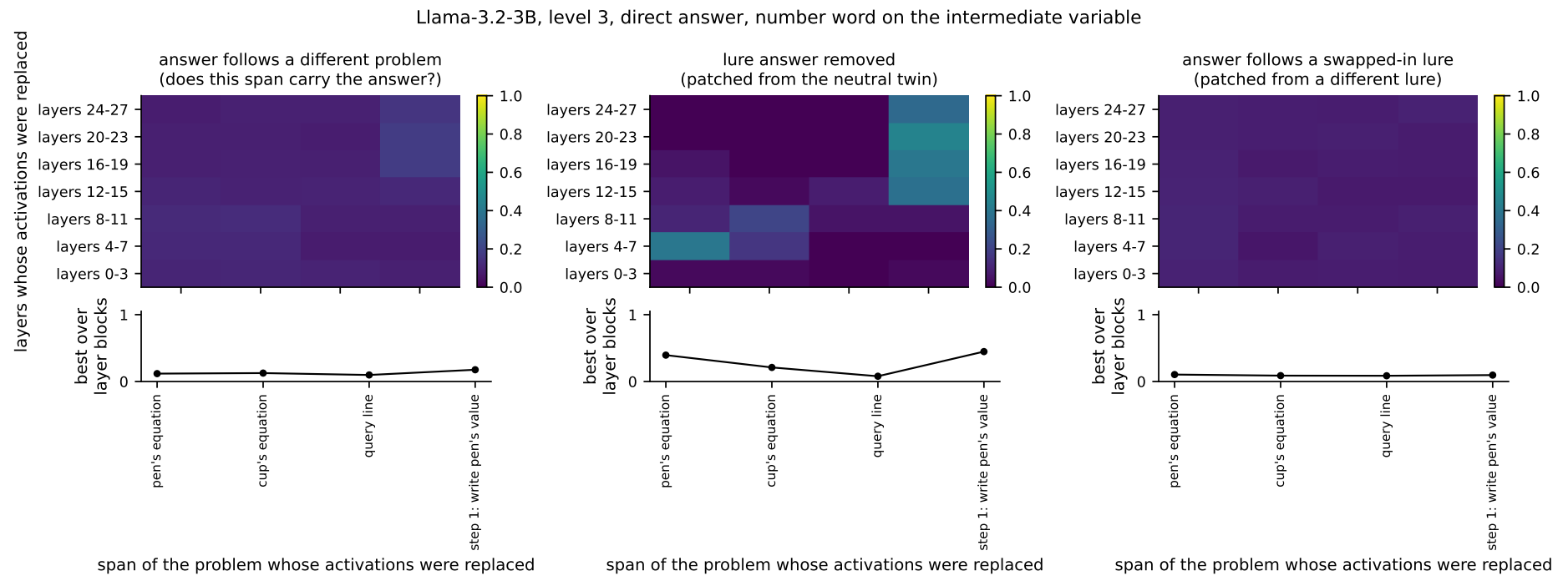


Figure 21: The same grids in the direct regime.

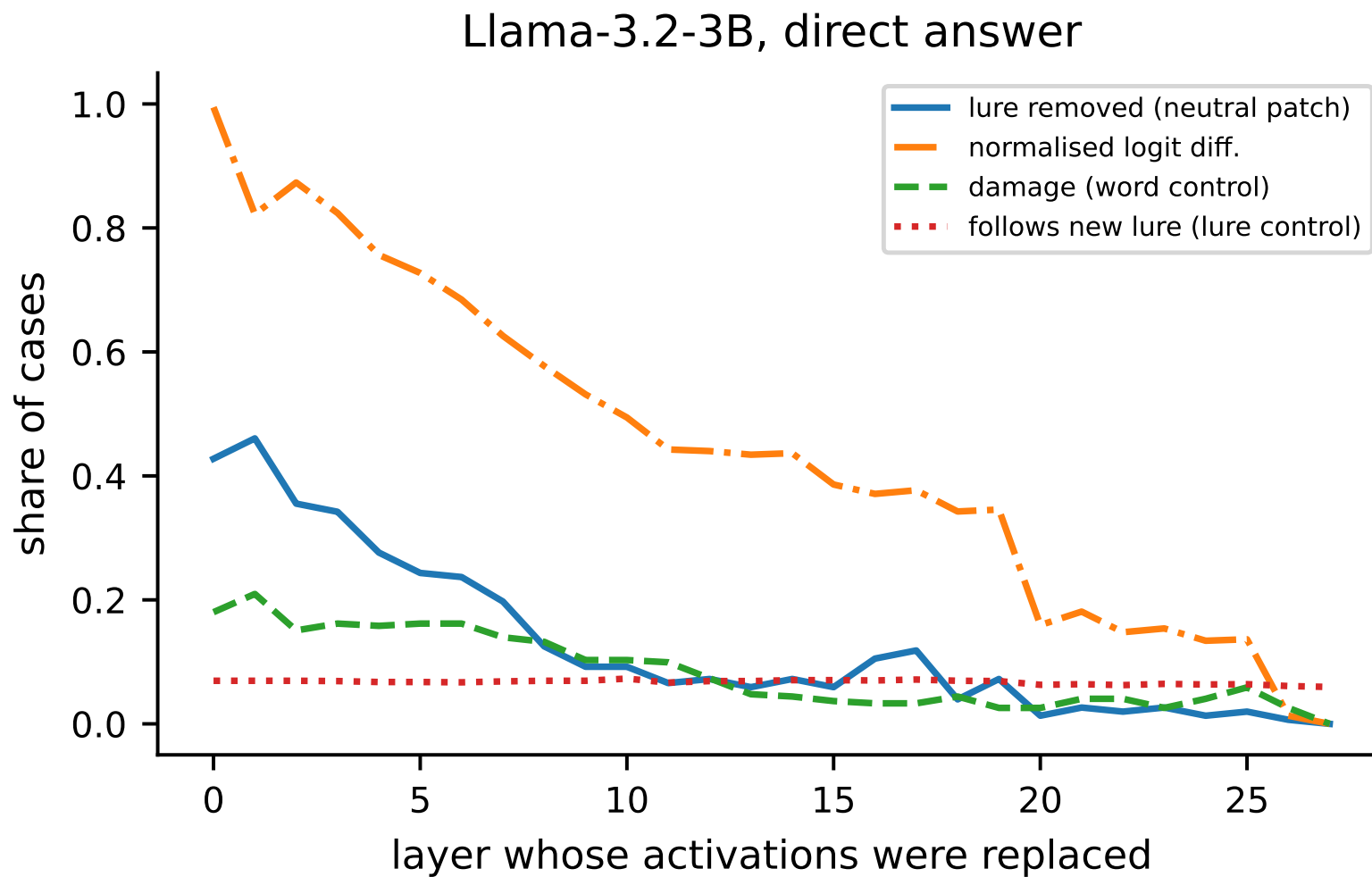


Figure 22: Lure errors removed by patching, by layer, with both controls.

llama31-8b, level 3, cot: per-token probes for the value of v1 (trained on train_neutral)

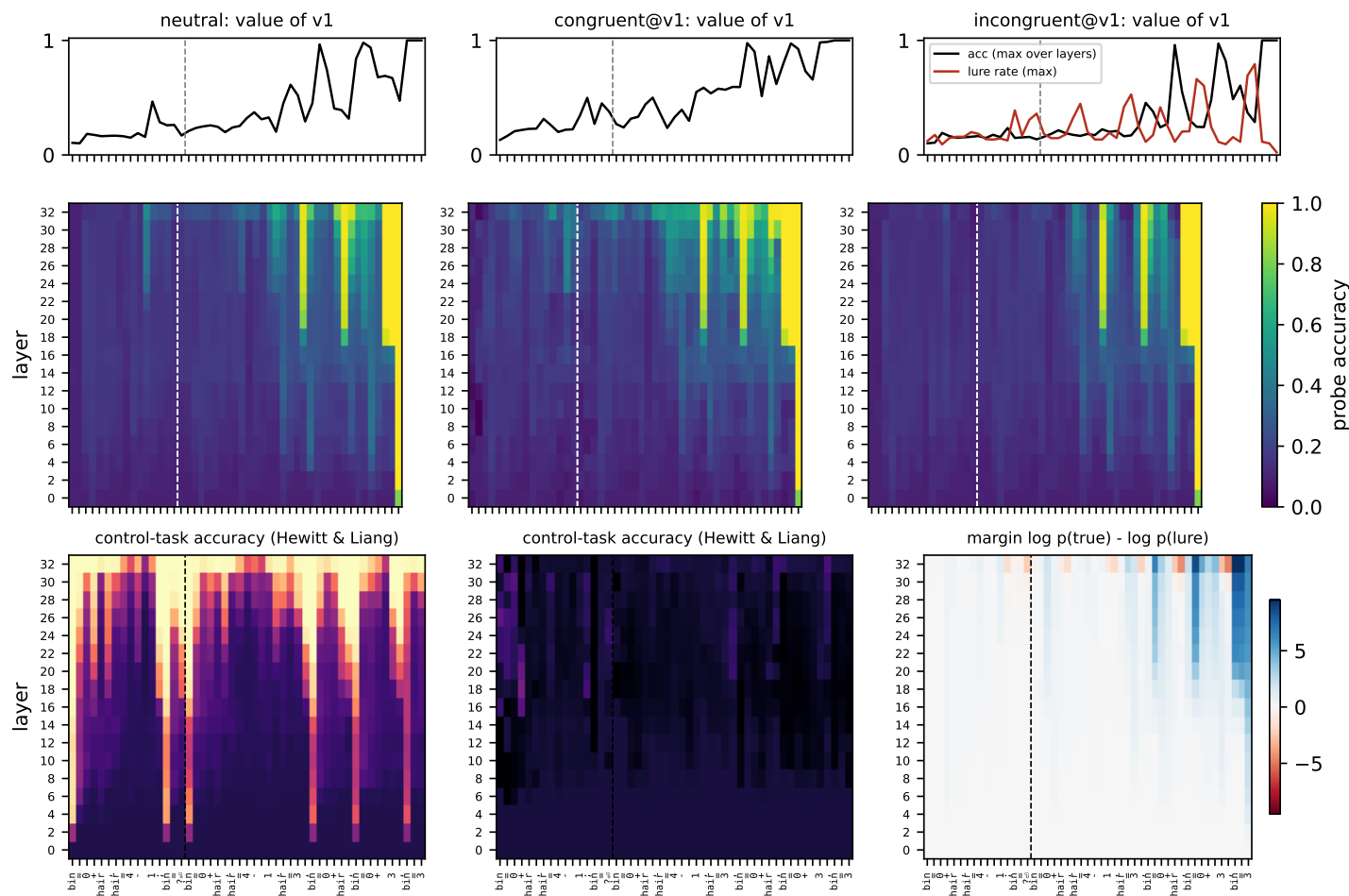


Figure 23: Llama-3.1-8B, chain-of-thought regime, queried variable: the accuracy heatmaps of the Llama-3.2-3B figures. The value is decodable from the equation that restates the variable's definition, before the chain writes it.

llama31-8b, level 3, cot: per-token probes for the value of v2 (trained on train_neutral)

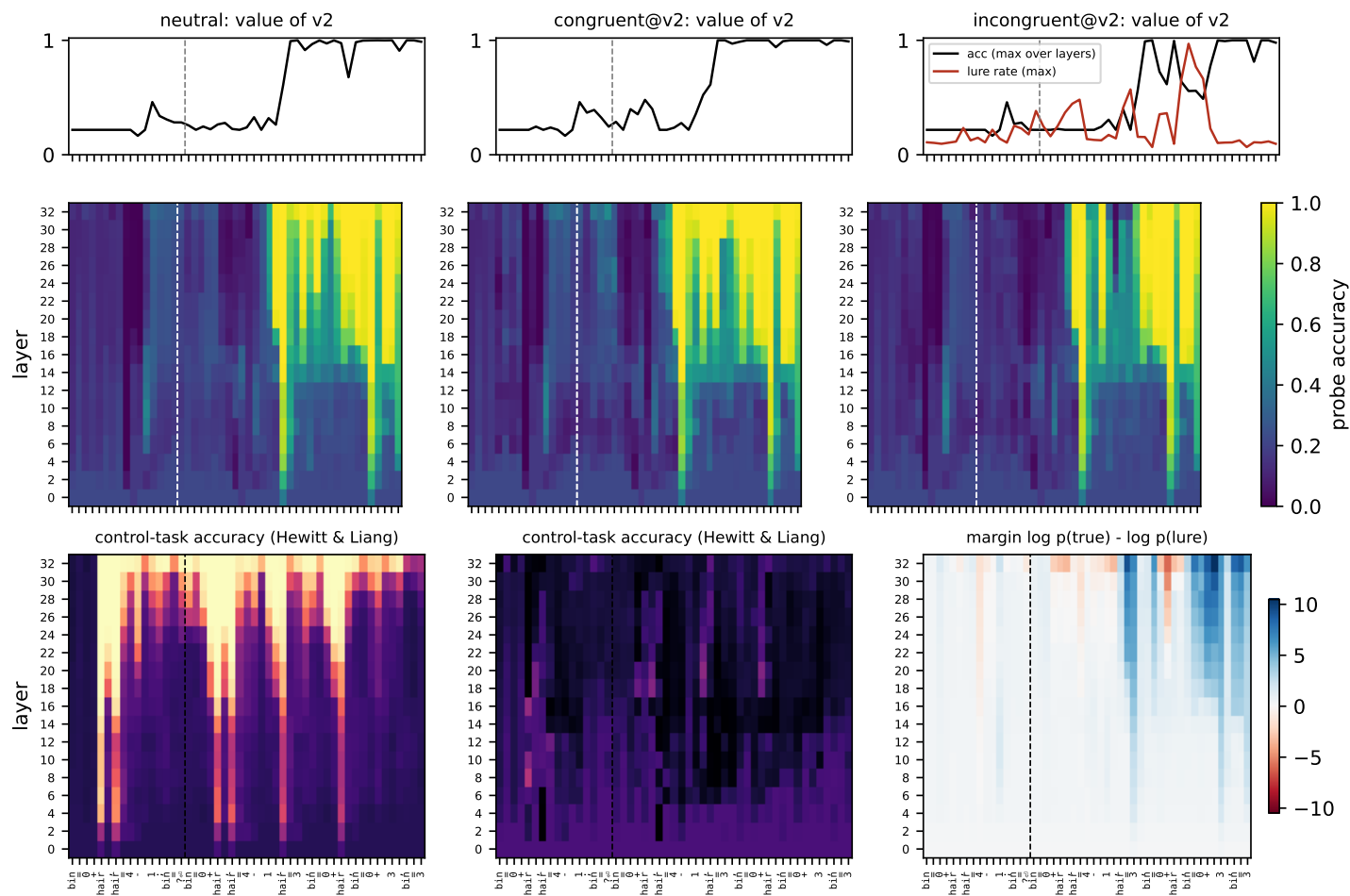


Figure 24: Llama-3.1-8B, chain-of-thought regime, intermediate variable.

llama31-8b, level 3, direct: per-token probes for the value of v2 (trained on train_neutral)

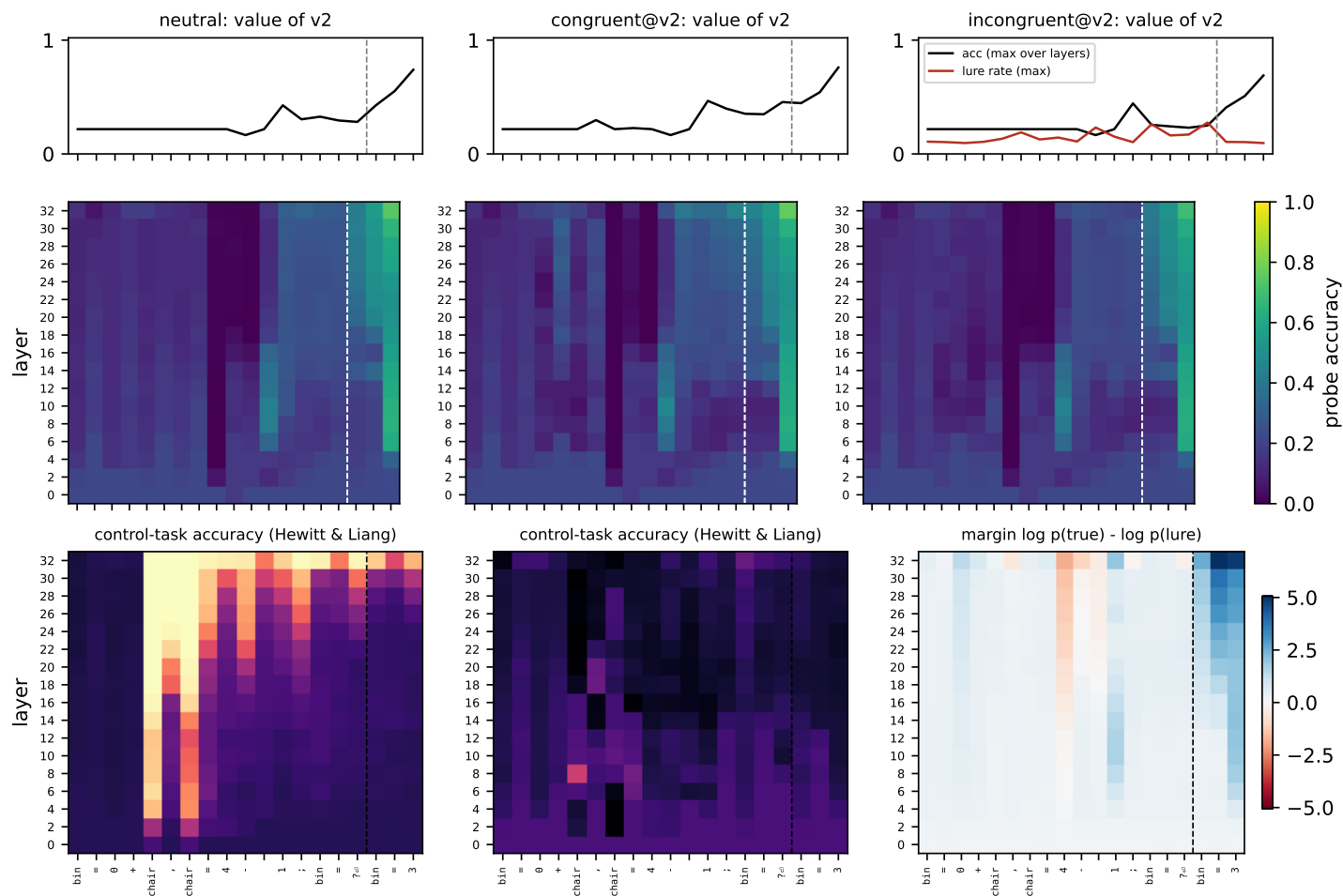


Figure 25: Llama-3.1-8B, direct regime, intermediate variable.

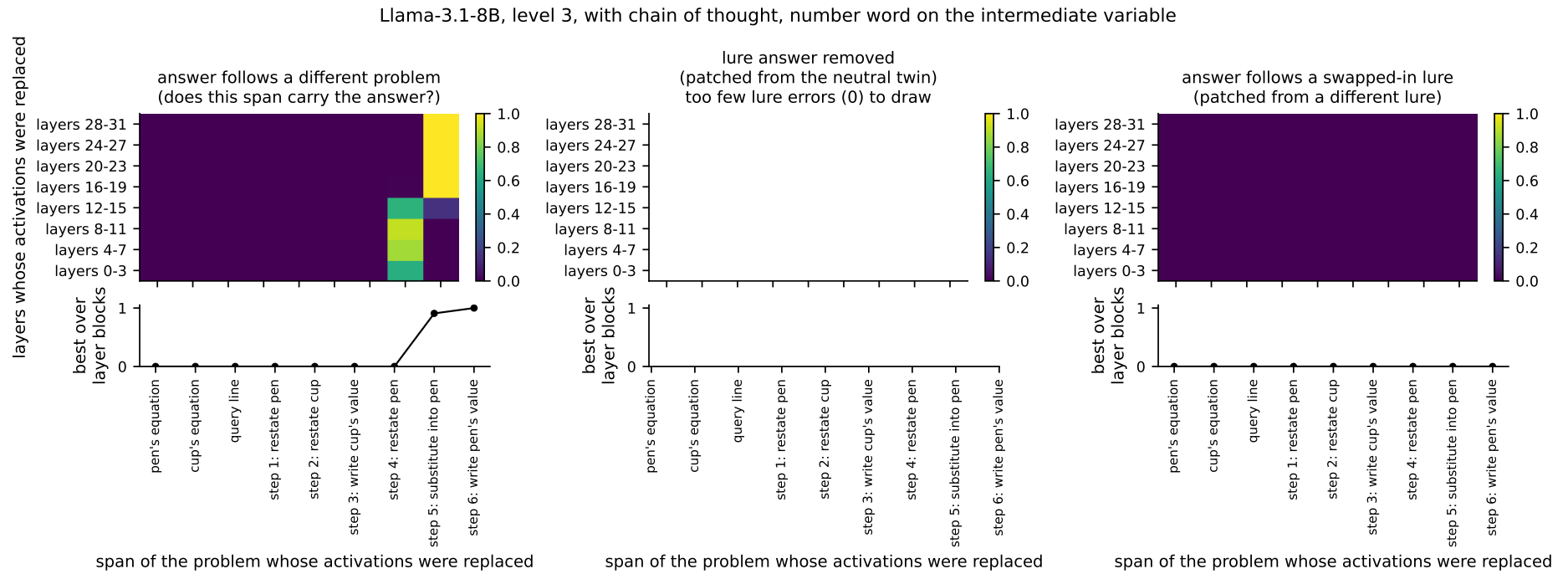


Figure 26: Llama-3.1-8B, span-by-layer-window patching grids, chain-of-thought regime. The lure-removal panel is empty: none of the 500 patched problems is answered with the lure.

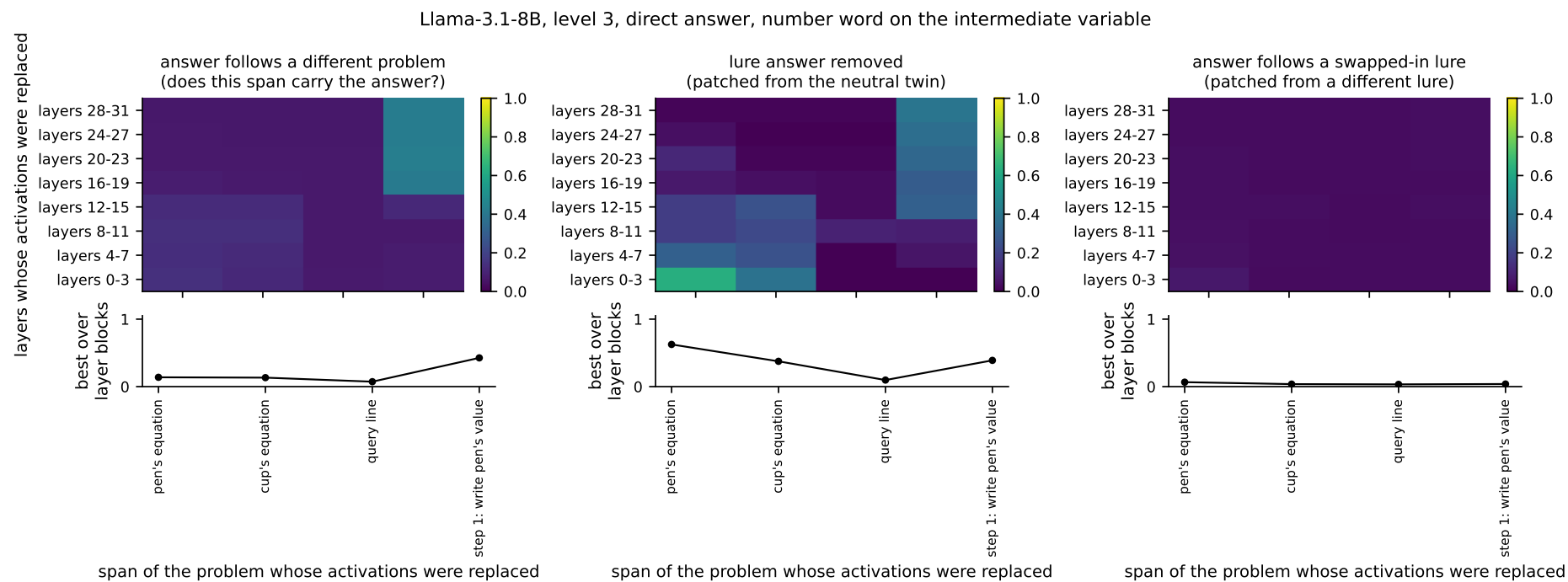


Figure 27: Llama-3.1-8B, the same grids in the direct regime.

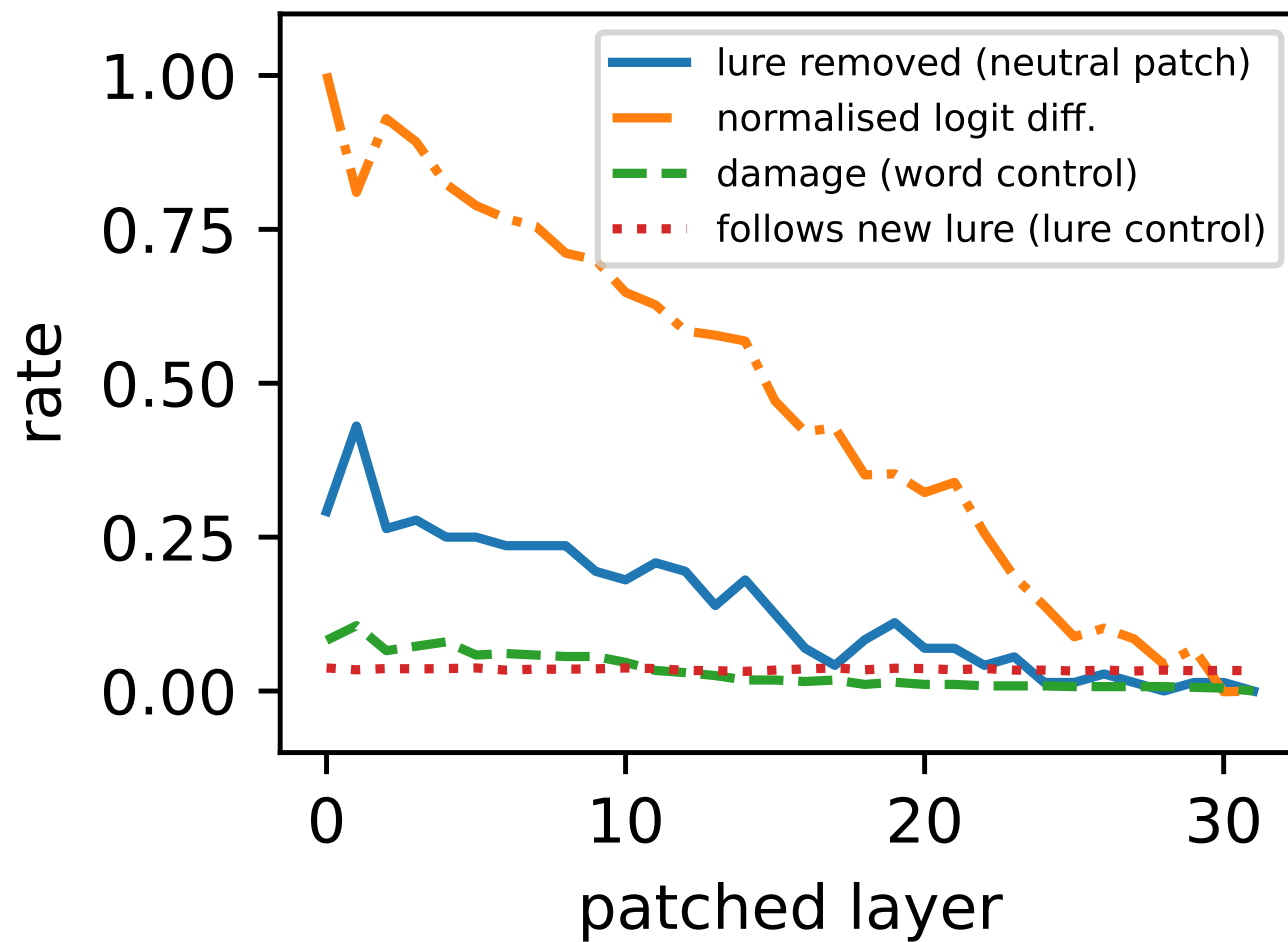


Figure 28: fig3_patching_llama31-8b_L3_v2.

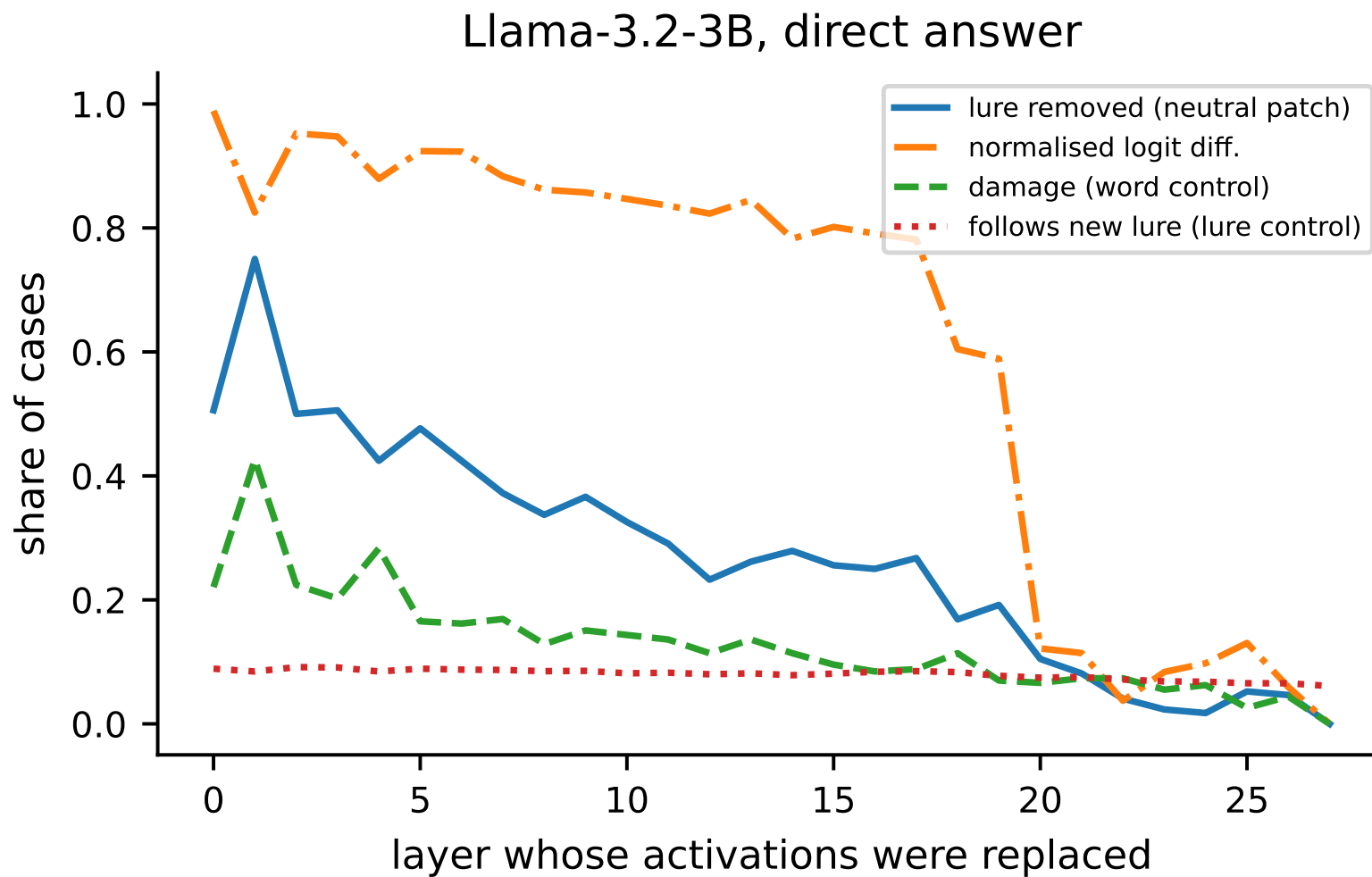
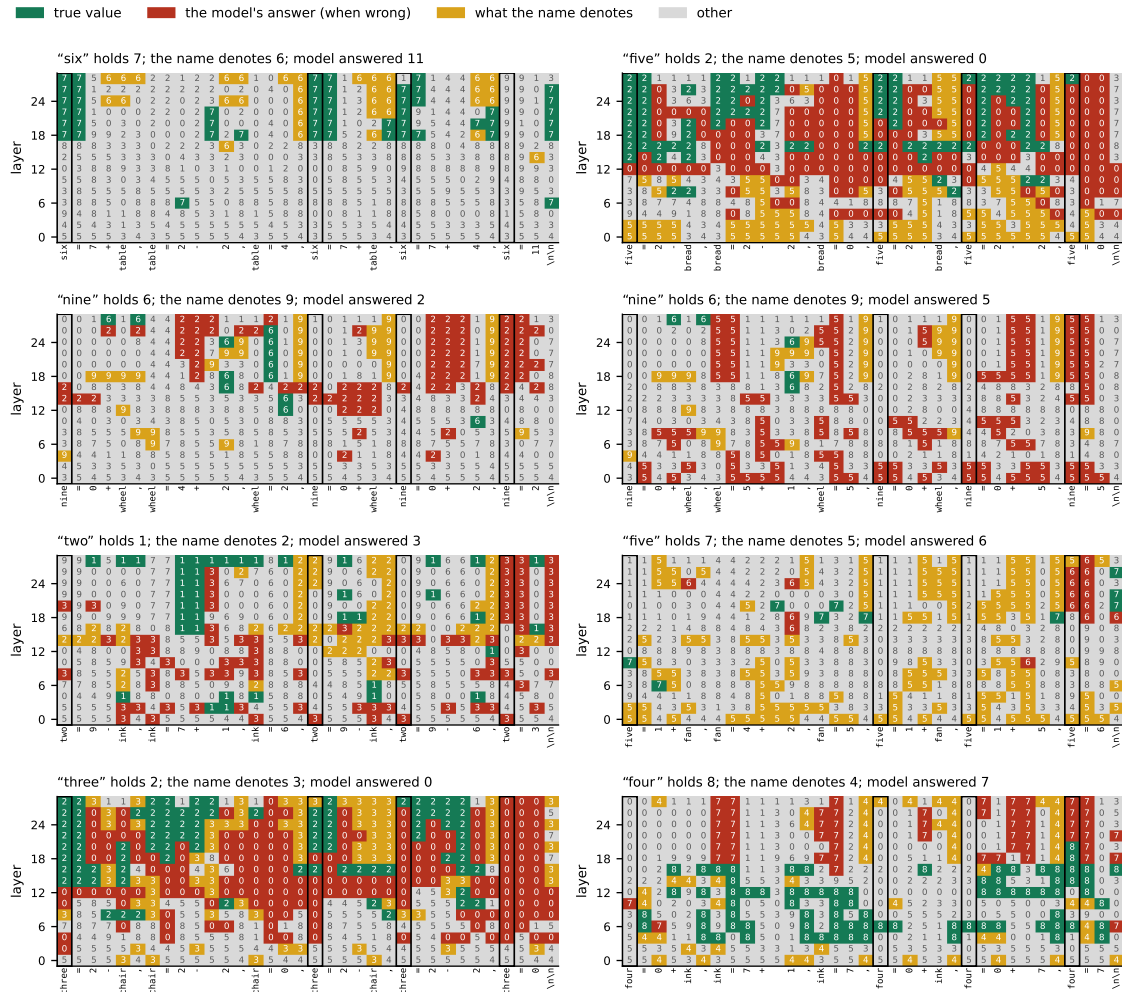


Figure 29: fig3_patching_llama32-3b_L3_v1.

Top-1 probe prediction over the model's own chain (Llama-3.2-3B, L3, with chain of thought; probes trained at the value step of "v1" on neutral problems; 8 instances)



llama31-8b, level 3, direct: per-token probes for the value of v1 (trained on train_neutral)

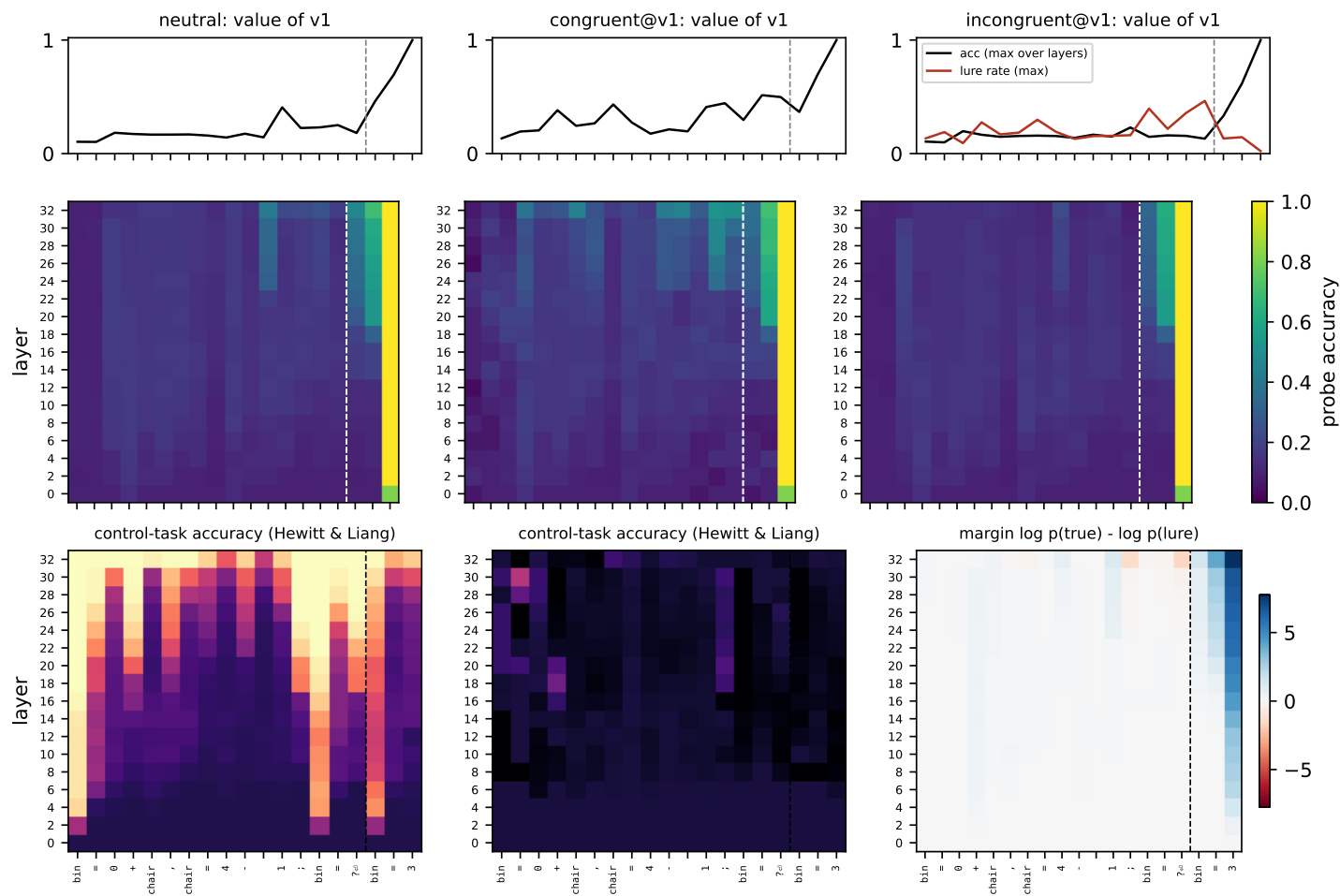


Figure 31: kudo_fig2_llama31-8b_L3_direct_v1.

llama32-3b, level 3, cot: per-token probes for the value of v1 (trained on train_letter)

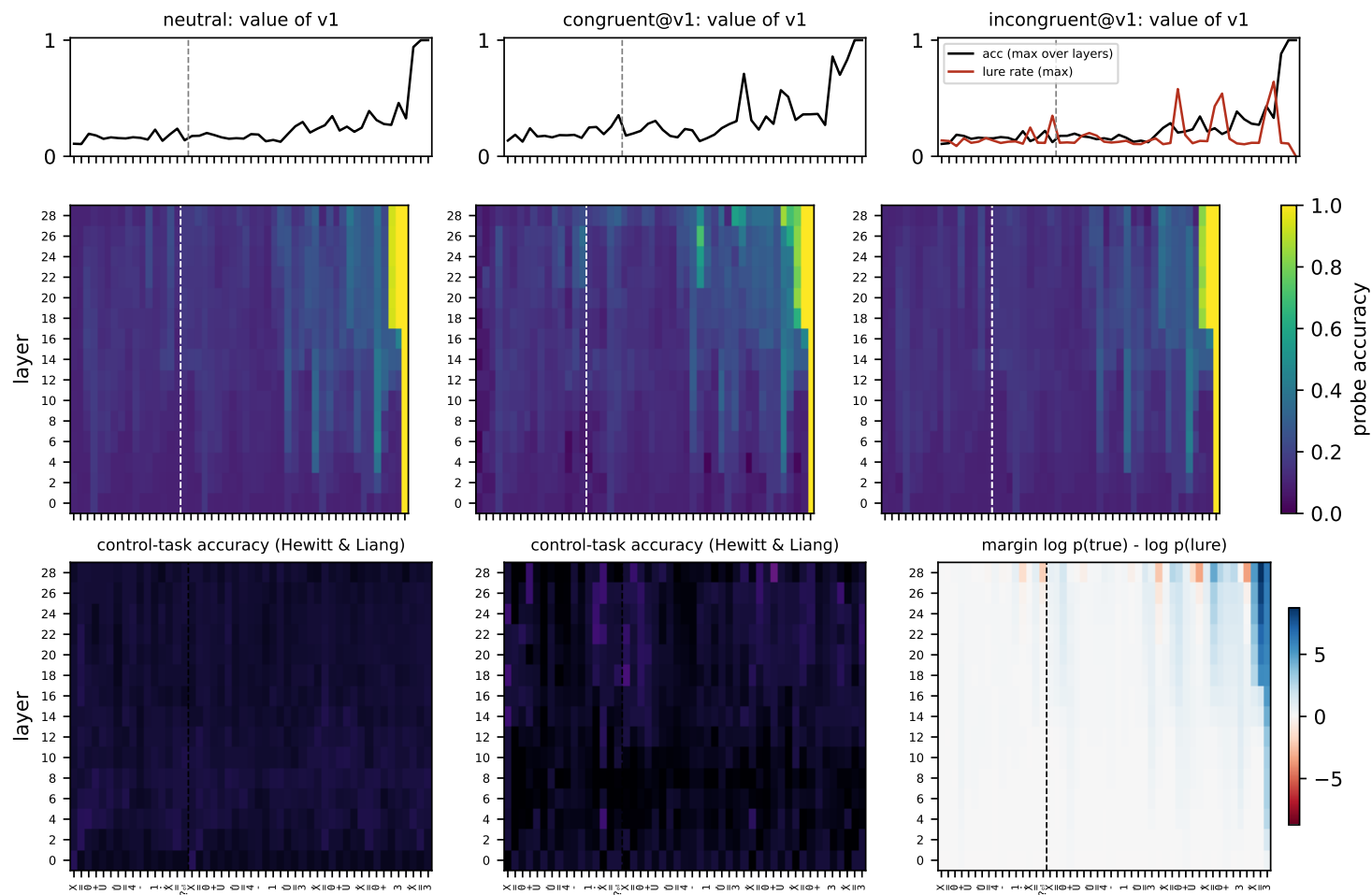


Figure 32: kudo_fig2_llama32-3b_L3_cot_v1.

llama32-3b, level 3, direct: per-token probes for the value of v1 (trained on train_neutral)

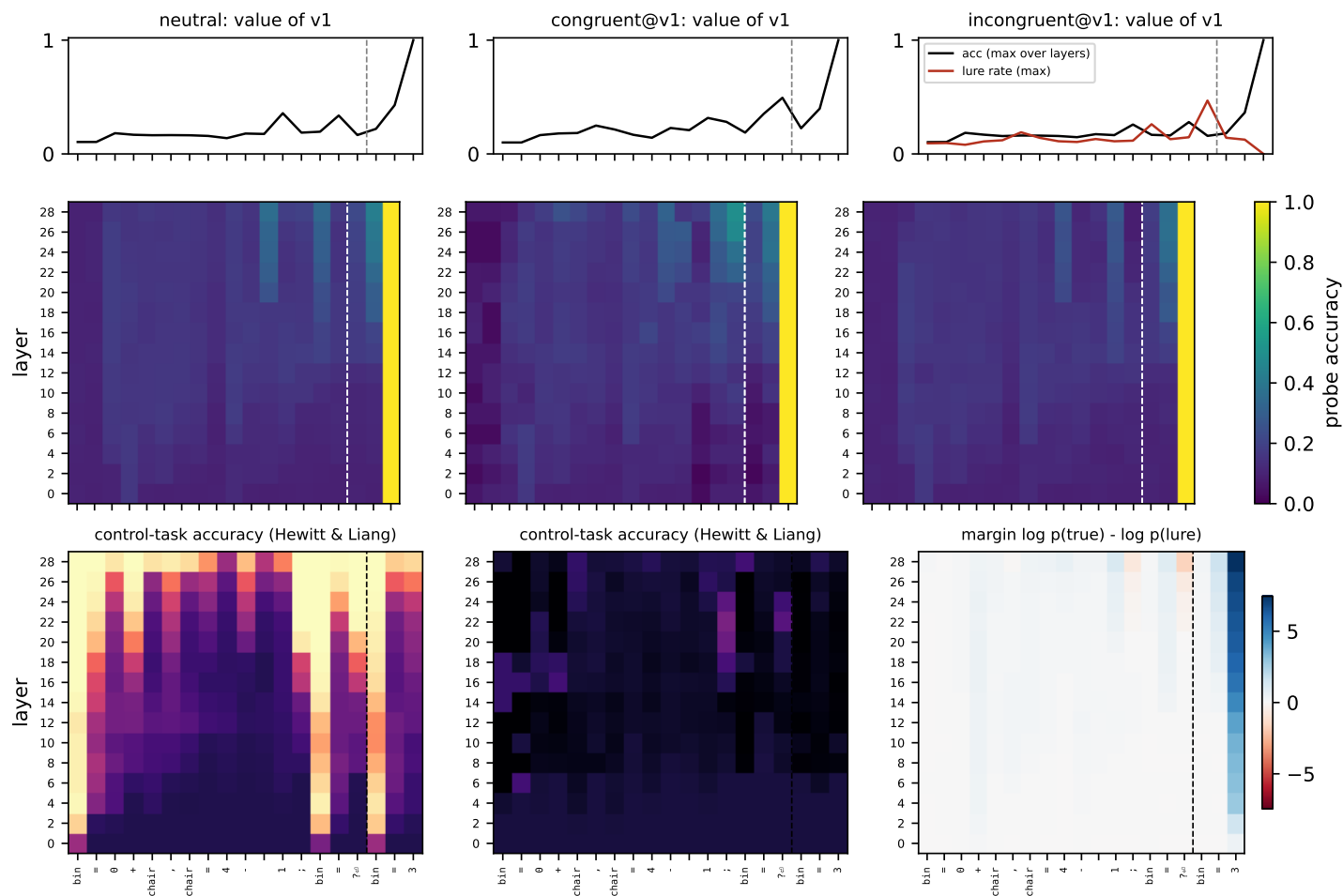


Figure 33: kudo_fig2_llama32-3b_L3_direct_v1.

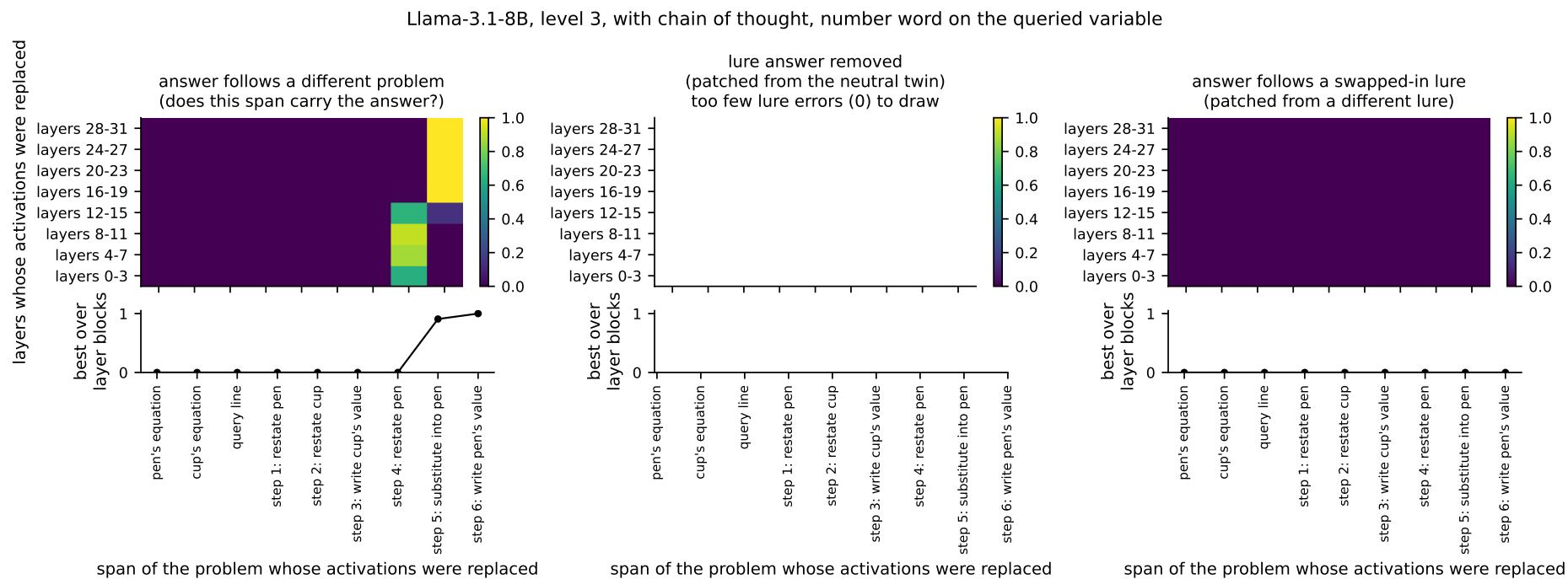


Figure 34: kudo_fig5_llama31-8b_L3_cot_v1.

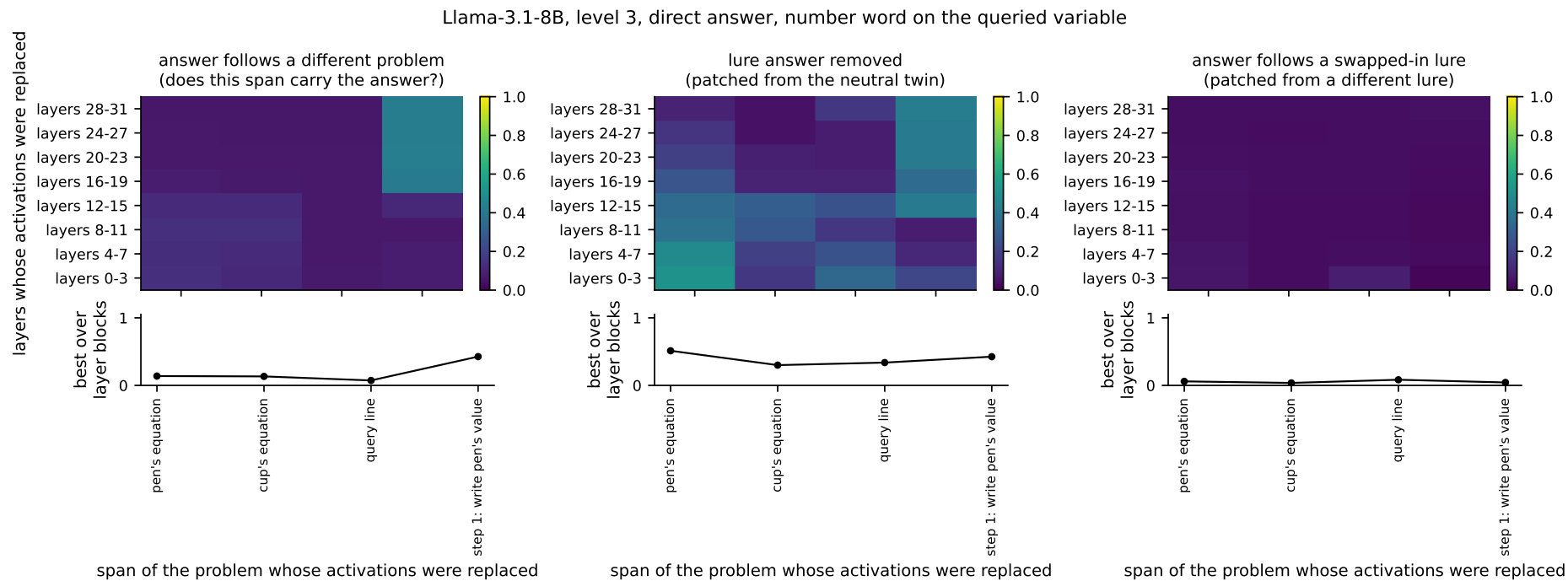


Figure 35: kudo_fig5_llama31-8b_L3_direct_v1.

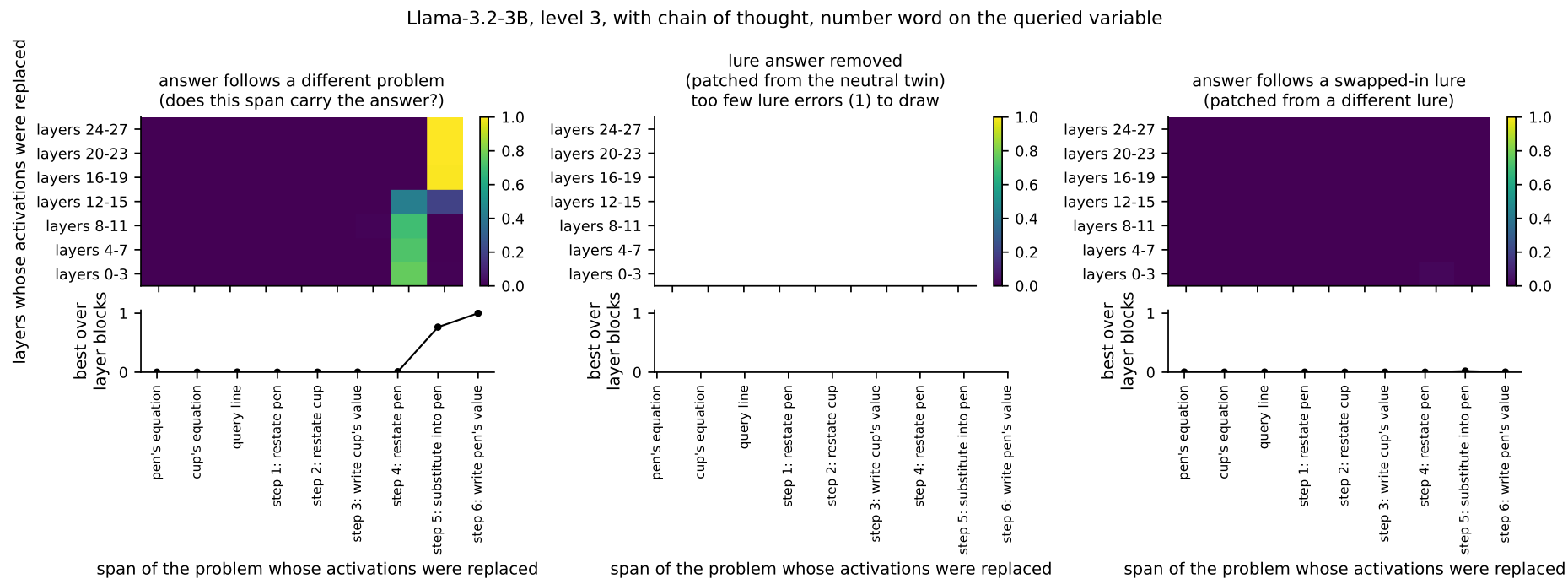


Figure 36: kudo_fig5_llama32-3b_L3_cot_v1.

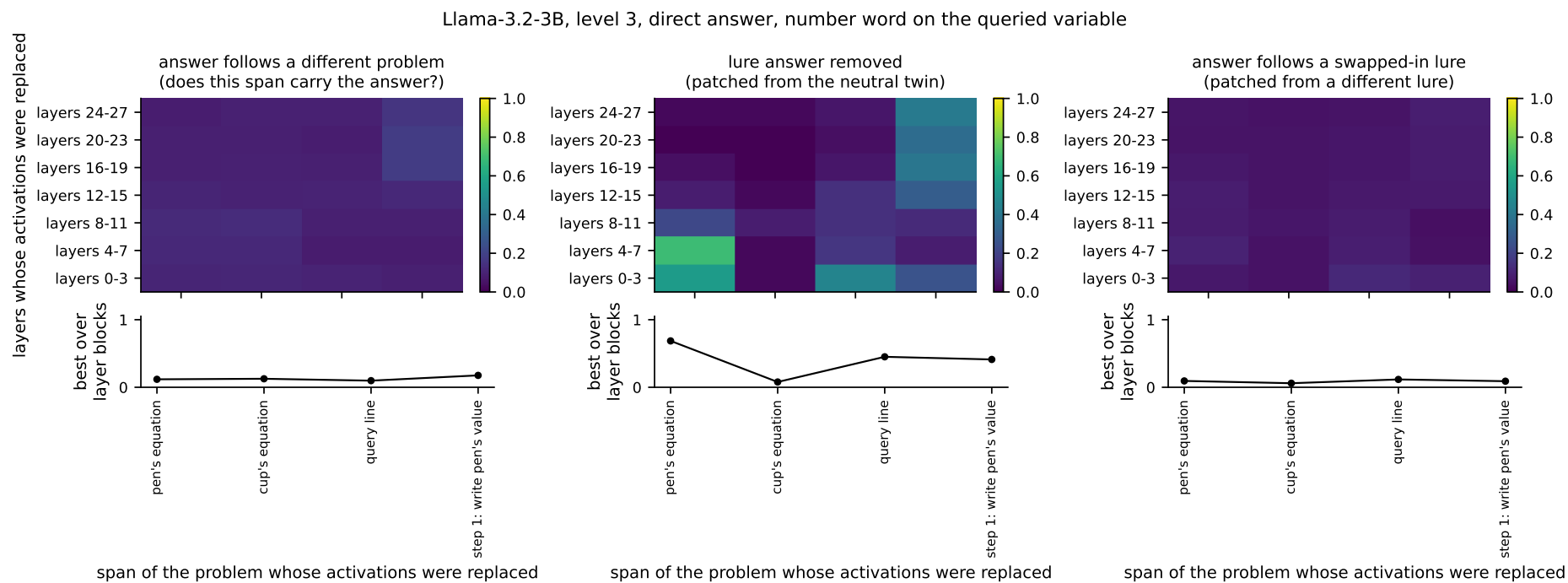


Figure 37: kudo_fig5_llama32-3b_L3_direct_v1.